

✓ Harmonic Oscillator

- **Author:** Boran Aybak Kilic
- **Date:** 04.26.2026
- **Time spent on this assignment:** 5 hours

Our goal in this assignment is to look at and think about the simple Harmonic oscillator.

```

1
2 import numpy as np
3 import scipy
4 import matplotlib.pyplot as plt
5 import math
6 from matplotlib.animation import FuncAnimation
7 from IPython.display import HTML
8 import numpy.polynomial.hermite as Herm
9 import math
10 import scipy.optimize
11
12
13 import matplotlib.animation as animation
14 from IPython.display import HTML
15 def resetMe(keepList=[]):
16     ll=%who_ls
17     keepList=keepList+['FuncAnimation','Herm','HTML','resetMe',
18     for iiii in keepList:
19         if iiii in ll:
20             ll.remove(iiii)
21     for iiii in ll:
22         jjjj="^"+iiii+"$"
23         %reset_selective -f {jjjj}
24     ll=%who_ls
25     plt.rcParams.update({"font.size": 14})
26     return
27 resetMe()
28 import datetime;datetime.datetime.now()

```

datetime.datetime(2026, 4, 27, 2, 25, 14, 843496)

In this assignment, we will consistently use grids and observables that we will set up in this way. This is very similar to how you set things for the particle in the box assignment.

```

1 def SetupGrid(L,delta_x):
2     n=int(round(L/delta_x))+1
3     xs=np.linspace(-L/2,L/2,n,endpoint=True)
4     return xs
5
6
7 def SetupObservables(xs,delta_x):
8     X=np.diag(xs)
9     P=(np.diag([-1.j/(2*delta_x) for i in range(len(xs)-1)],k=1
10
11     P2=np.zeros_like(P)
12     for i in range(len(xs)):
13         P2[i,i]=2.0/delta_x**2
14         if i+1<len(xs):
15             P2[i,i+1]=-1.0/delta_x**2
16             P2[i+1,i]=-1.0/delta_x**2
17     return X,P,P2
18
19
20
21 def update(frame, skip, xs, positions=None, potential=[], max_v
22     plt.cla() # Clear the current plot
23     scale=max_value/energy
24     plt.axhline(energy*scale, color='red', linestyle='--')
25     plt.plot(xs,potential*scale)
26     if type(positions)==np.ndarray:
27         plt.axvline(positions[::skip][frame], color='green', li
28     elif positions!=None:
29         print("ERROR")
30         return
31     plt.plot(xs, np.abs(arrays[::skip][frame])**2) # Plot the
32     plt.ylim(0, 1.5*max_value) # Set the y-axis limit
33     plt.xlabel('x')
34     plt.ylabel('Value')
35     plt.title(f'Frame {skip*frame+1}/{len(arrays)}') # Display
36
37

```

✓ Exercise 1. The Uncertainty Principle

In this exercise, we'd like to think about the uncertainty principle.

✓ a. The Fourier Basis

The uncertainty principle tells us that you can't simultaneously know a particle's position and momentum. It's typically quantified as

$$\Delta X \Delta P > \frac{1}{2}$$

Let's think for a moment about what it means to be certain or uncertain about a particles position. If you are uncertain about the position of a particle, what this means is that if you measure that particle many times, you get very different answers for the x position of a particle. If you are certain about the position of a particle this means that most measurements come out concentrated in a particular region so even before measuring, we could feel reasonably certain where the particle is in x .

Here is an example of a wave-function which you are reasonably certain about it's position:

$$\Psi = N \exp[-\alpha(x - 4)^6]$$

where N is the normalization and $\alpha = 100$.

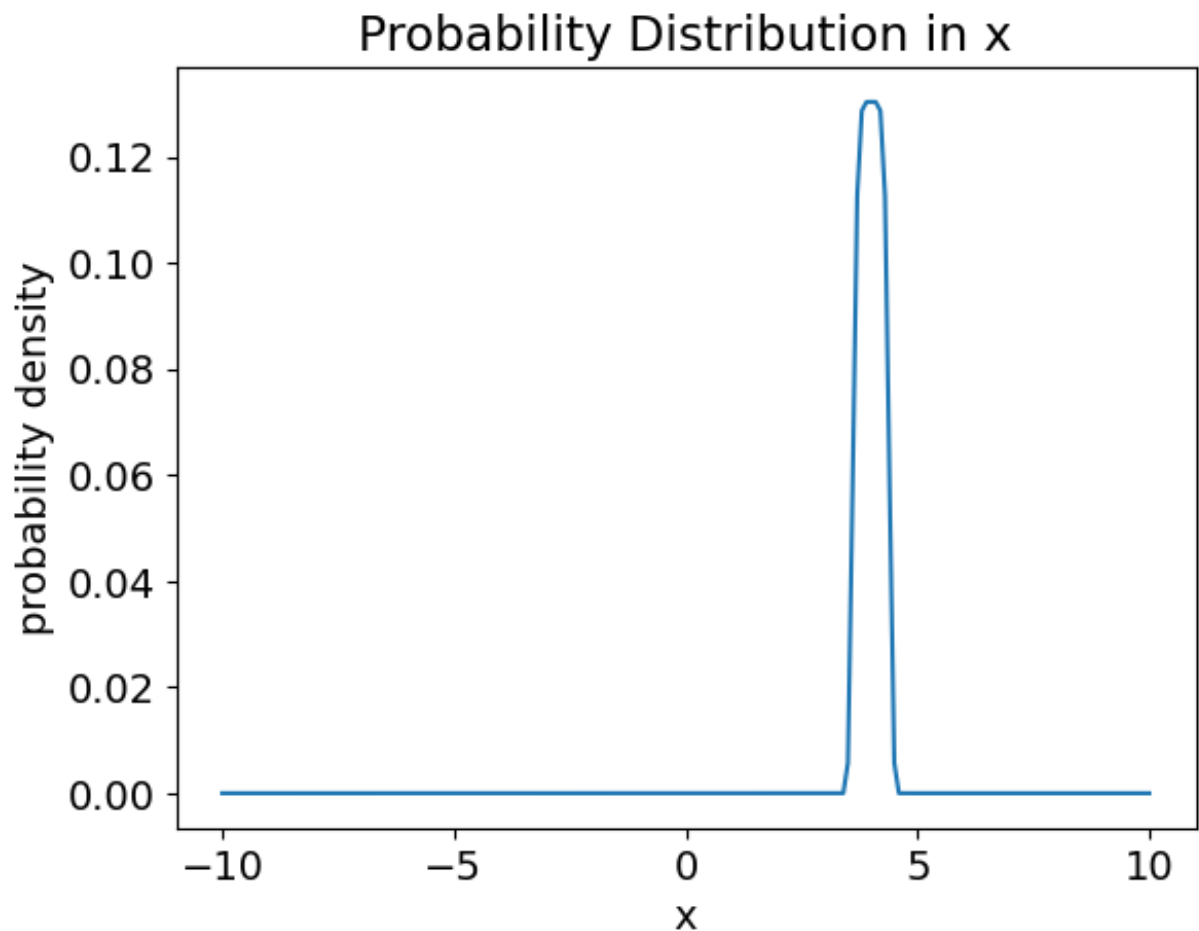
After setting up your grid and observables:

```
delta_x=0.1
xs=SetupGrid(20,delta_x)
X,P,P2=SetupObservables(xs,delta_x)
```

go ahead and plot the probability (absolute value squared of the wave-function) of getting different x values for this wave-function remembering to label the x -axis as `plt.xlabel("x")` and the y -axis as `plt.ylabel("probability density")`. From this plot, you should be able to see that if you measure x over and over again it will always be getting an answer somewhat around 4.

? Answer (start)

```
1 alpha = 100
2 delta_x=0.1
3 xs=SetupGrid(20,delta_x)
4 X,P,P2=SetupObservables(xs,delta_x)
5 psi = np.exp(-1*alpha*(xs-4)**6)
6 psi=psi/np.linalg.norm(psi)
7 psi_squared=np.abs(psi)**2
8 plt.plot(xs,psi_squared)
9 plt.xlabel("x")
10 plt.ylabel("probability density")
11 plt.title("Probability Distribution in x")
12 plt.show()
```



✓ Answer (end)

We are now hoping that there's also a pretty narrow range of where the particle can be in momentum P - i.e. if we measure the momentum over and over again would we get essentially the same value. It's hard to look at our plot and figure this out at the moment. That's because the wave-function is currently plotted in the x -basis. We could also plot it in the momentum basis. If we did that, then we could look at the wave-function squared and see how spread out it is in momentum (and so whether we would expect to get the same momentum if we measured it).

So we'd like to replot the wave-function as a function of p and not x . Mind you, this is the same wave-function. It's just a different way of representing it. If I show you a curve and I don't tell label the axis so you know whether you are in the x basis or p basis, you don't really know what you're looking at.

To change a basis you multiply by a unitary matrix that corresponds to that basis change. We need the unitary matrix that multiplies a wave-function in the x -basis and converts it to the p -basis. Then we can plot it in the p -basis. You can get this unitary (along with the basis of p -points) by doing

```
def MomentumBasis(L,delta_x):
    N=int(round(L/delta_x))+1
    ns=np.array(range(0,N))
    xs=np.linspace(-L/2,L/2,N,endpoint=True)
    ks=np.array(ns*2*np.pi)/(N*delta_x)
    U=1.0/np.sqrt(N)*np.exp(-1.j*np.outer(xs,ks))
    return ks,U
```

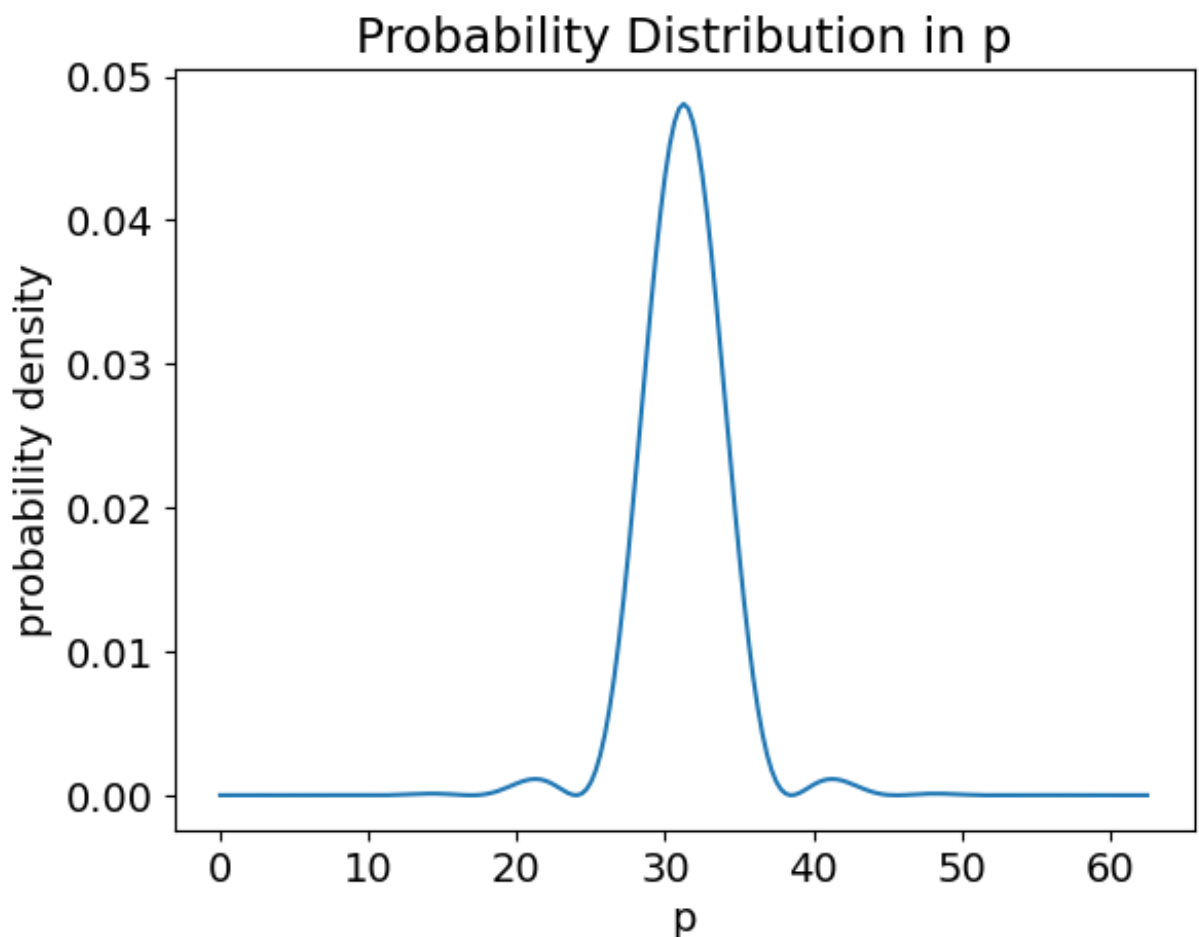
Now by getting out this unitary (`(ps,U=MomentumBasis(20,0.1))`) you can then apply `U@psi` to rotate into the fourier basis. Take the wave-function in real space and rotate it into the fourier basis. Plot the wave-function squared in the momentum basis and see if (after measurement) in the momentum basis you are likely to get consistent answers for the momentum.

? Answer (start)

```

1 def MomentumBasis(L,delta_x):
2     N=int(round(L/delta_x))+1
3     ns=np.array(range(0,N))
4     xs=np.linspace(-L/2,L/2,N,endpoint=True)
5     ks=np.array(ns*2*np.pi)/(N*delta_x)
6     U=1.0/np.sqrt(N)*np.exp(-1.j*np.outer(xs,ks))
7     return ks,U
8
9 ps,U=MomentumBasis(20,0.1)
10 psi_p=U@psi
11 psi_p_squared=np.abs(psi_p)**2
12 plt.plot(ps,psi_p_squared)
13 plt.xlabel("p")
14 plt.ylabel("probability density")
15 plt.title("Probability Distribution in p")
16 plt.show()

```



✓ Answer (end)

While our position resolution is good, our momentum resolution is not that great - it seems we can locate the momentum to within 5'sh.

✓ b. Measuring Uncertainty

To get a better handle on this, we want to be able to quantify the uncertainty. The measure of certainty essentially needs to be a measure of the spread of the wavefunction which can be quantified by the variance of the wavefunction in the position or momentum basis - i.e.

- $\Delta X = \sqrt{\langle X^2 \rangle - \langle X \rangle^2}$
- $\Delta P = \sqrt{\langle P^2 \rangle - \langle P \rangle^2}$

Recall that you can get the expectation value of an observable O by doing

```
psi.conjugate() @ O @ psi.T
```

Write a function `def uncertainty(psi,X,P)` which returns the ΔX and ΔP and $\Delta X \Delta P$.

Now compute the uncertainty for the wave-functions above. You should quantify what we say above. The variance of the position is smallish but the variance of the momentum is much larger and so if we measured we would be getting widely different momentum.

? Answer (start)

```
1
2 si.T
3 si.T
4 @ psi.T
5 psi.T
6 ?)
7 ?)
8
9
10 ,X,P)
11 } \nUncertainty of p is {np.real(d_p)} \nDelta(X).Delta(P):{np.
```

```
Uncertainty of x is 0.23331078991742396
Uncertainty of p is 2.8941417673773246
Delta(X).Delta(P):0.675234501879813
```

✓ Answer (end)

Our wave-function does have a tuning parameter α that will make it more or less concentrated in x (for example try $\alpha = 0.0001$ in the x -basis). Write some code to tune α from 10 down to 0.0001. On the same plot, show as a function of α

- the value of ΔX
- the value of ΔP
- the value of $\Delta X \Delta P$

What is the minimum value of $\Delta X \Delta P$ you can find. The uncertainty principle tells us that $\Delta X \Delta P > 0.5$. What's the closest you get?

(you may find it useful here to work on a logarithmic grid of alpha - i.e.

```
alphas=np.arange(0.0001,10.01,0.01)
alphas=np.exp(np.arange(-10,10,1.0))
```

If you do this, then you are going to want to change the x-axis on a log-scale as

```
plt.xscale('log')
```

Plot also the theoretical minima `plt.ahline(0.5)`

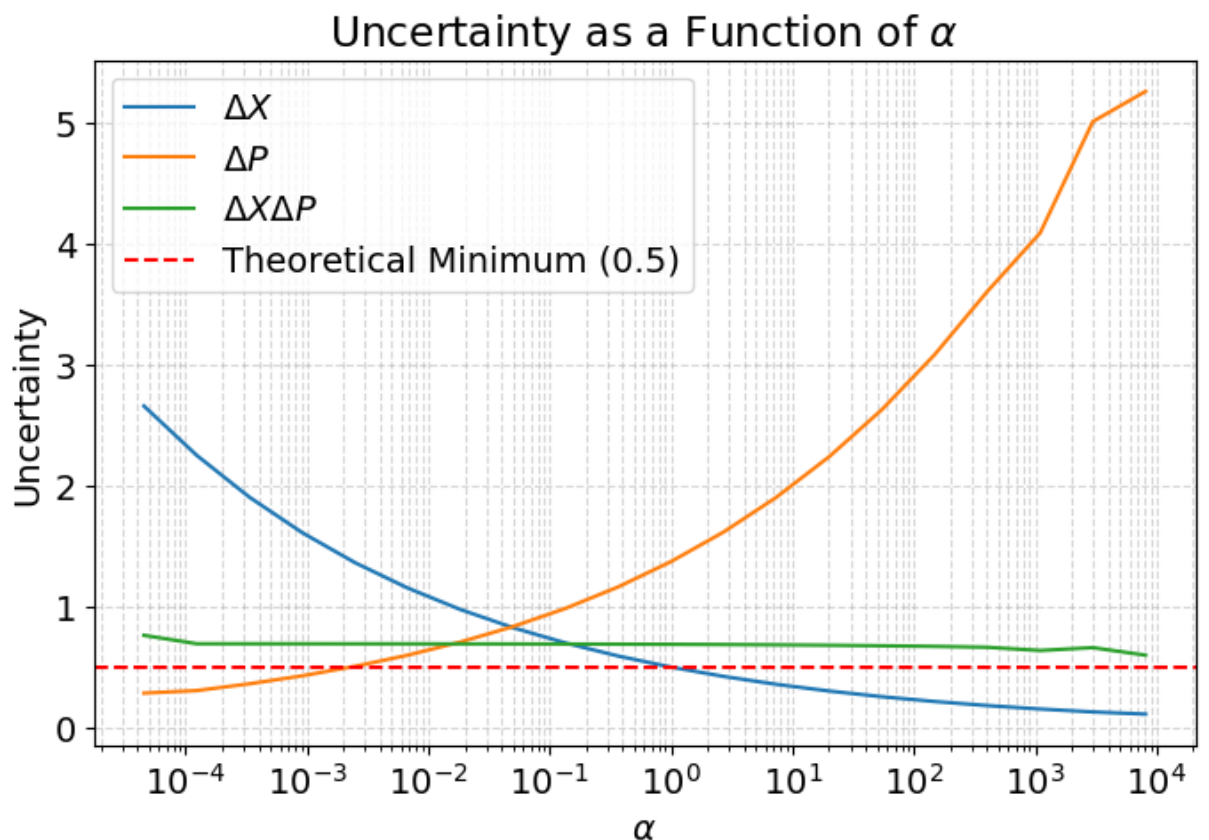
? Answer (start)

```
1 alphas = np.exp(np.arange(-10, 10, 1.0))
2 delta_x_list = []
3 delta_p_list = []
4 delta_xp_list = []
5
6 for alpha in alphas:
7     # Generate wave-function for current alpha
8     psi = np.exp(-1 * alpha * (xs - 4)**6)
9     psi = psi / np.linalg.norm(psi)
10
11     # Calculate uncertainties
12     dx, dp, dxp = uncertainty(psi, X, P)
13
14     delta_x_list.append(np.real(dx))
15     delta_p_list.append(np.real(dp))
16     delta_xp_list.append(np.real(dxp))
17
18 # Visualization
```

```

19 plt.figure(figsize=(8, 5))
20 plt.plot(alphas, delta_x_list, label=r'$\Delta X$')
21 plt.plot(alphas, delta_p_list, label=r'$\Delta P$')
22 plt.plot(alphas, delta_xp_list, label=r'$\Delta X \Delta P$')
23 plt.axhline(0.5, color='r', linestyle='--', label='Theoretical')
24
25 plt.xscale('log')
26 plt.xlabel(r'$\alpha$')
27 plt.ylabel('Uncertainty')
28 plt.title(r'Uncertainty as a Function of $\alpha$')
29 plt.legend()
30 plt.grid(True, which='both', linestyle='--', alpha=0.5)
31 plt.show()
32
33 print(f"Minimum Delta(X)Delta(P): {min(delta_xp_list):.4f} achieved at alpha={min_index}")

```



Minimum Delta(X)Delta(P): 0.6012 achieved at alpha=8103.083927575384

✓ Answer (end)

✓ c. Minimizing Uncertainty

So far we've been having trouble finding something that is compact both in real space and in momentum space. In particular, we haven't found a wave-function that matches the Heisenberg limit. In this section, we are going to try to optimize for the minimum uncertainty. (*you want to do this section with a grid spacing of $dx=0.1$*).

We will perform this optimization using python's `scipy.optimize`. To do this, we need to write an objective function `def f(psi)` that takes the wave-function `psi` and returns the thing we want to minimize.

You could do this by calling your `uncertainty` function and just returning $\Delta X \Delta P$. It will be useful to add to our objective function a small penalty for cases where either ΔX or ΔP are very large (we want to avoid this because we want to know x and p both pretty well and this helps the optimization because there is a subtle issue with our discretization as $\Delta X \rightarrow 0$ and ΔP gets very large).

Therefore, use an objective function $\Delta X \Delta P + 0.01 \max(\Delta X, \Delta P)$. Make sure your three delta are all real.

Now we can call the optimization function. The optimization function needs to be sent out objective function and a guess - i.e.

`ans=scipy.optimize.minimize(f,guess)`. You can use as your guess $\sin(0.3141592653589793x)$ which you can check is not very localized.

Once the optimizations is done, the new wave-function is `ans.x`. Check the uncertainty of your new wave-function and plot it both in the x-basis and the p-basis. You may notice that your wave-function looks gaussian. You can double check this by trying to fit/plot a gaussian on top of it.

? Answer (start)

```

1 def f(psi_guess):
2     # Ensure the wavefunction is normalized
3     psi_norm = psi_guess / np.linalg.norm(psi_guess)
4     dx, dp, dxp = uncertainty(psi_norm, X, P)
5     # Objective: dxp + 0.01 * max(dx, dp)
6     return np.real(dxp) + 0.01 * max(np.real(dx), np.real(dp))
7
8 # Initial guess using a sine wave
9 guess = np.sin(0.3141592653589793 * xs)
10
11 # Perform optimization
12 ans = scipy.optimize.minimize(f, guess)
13 psi_opt = ans.x
14 psi_opt = psi_opt / np.linalg.norm(psi_opt)
15
16 # Check the uncertainty of the new wavefunction
17 dx, dp, dxp = uncertainty(psi_opt, X, P)
18 print(f"Optimized Uncertainty: delta_X = {np.real(dx):.4f}, del

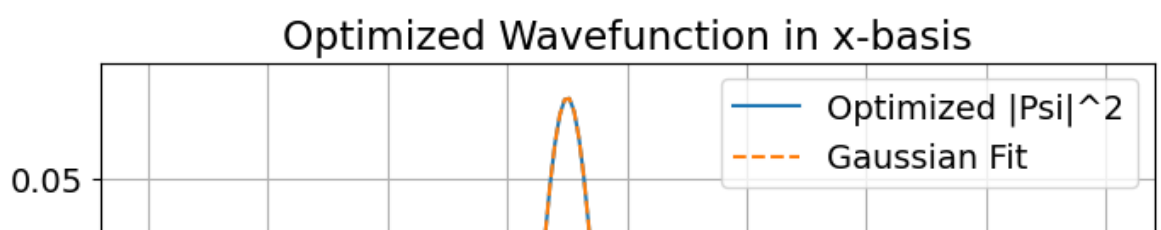
```

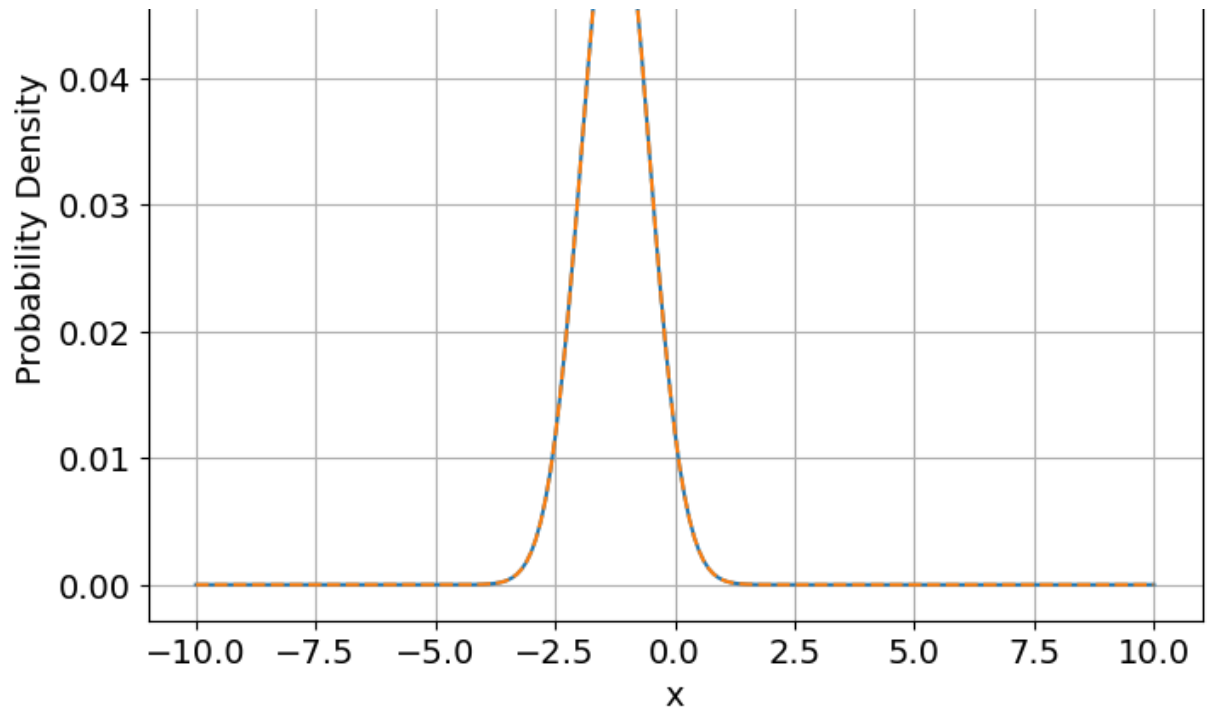
Optimized Uncertainty: delta_X = 0.7069, delta_P = 0.7069, delta_X*c

```

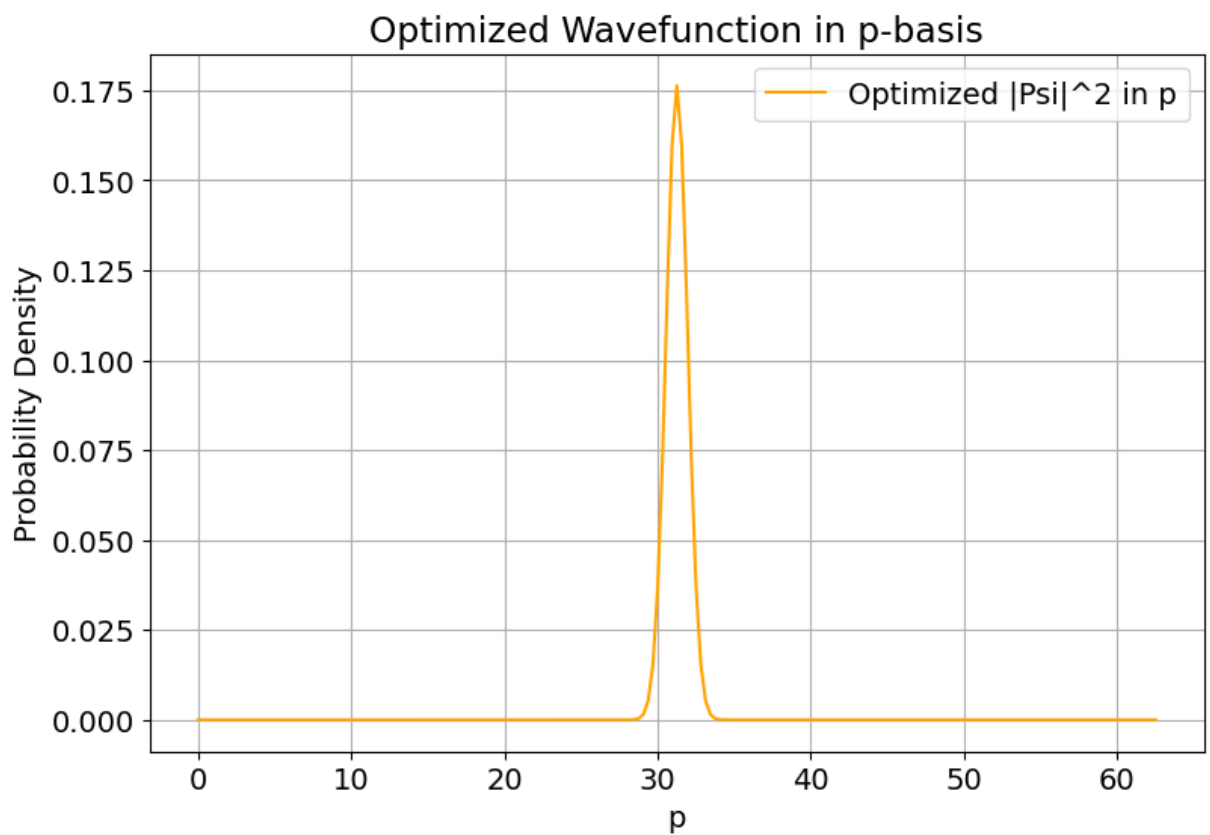
1 from scipy.optimize import curve_fit
2
3 psi_opt_sq = np.abs(psi_opt)**2
4
5 def gauss(x, a, x0, sigma):
6     return a * np.exp(-(x - x0)**2 / (2 * sigma**2))
7
8 # Fit Gaussian
9 popt, _ = curve_fit(gauss, xs, psi_opt_sq, p0=[max(psi_opt_sq),
10
11 plt.figure(figsize=(8, 6))
12 plt.plot(xs, psi_opt_sq, label="Optimized |Psi|^2")
13 plt.plot(xs, gauss(xs, *popt), '--', label="Gaussian Fit")
14 plt.xlabel("x")
15 plt.ylabel("Probability Density")
16 plt.legend()
17 plt.title("Optimized Wavefunction in x-basis")
18 plt.grid(True)
19 plt.show()

```





```
1 ps, U = MomentumBasis(20, delta_x)
2 psi_p = U @ psi_opt
3 psi_p_sq = np.abs(psi_p)**2
4
5
6 plt.figure(figsize=(9, 6))
7 plt.plot(ps, psi_p_sq, label="Optimized  $|\Psi|^2$  in p", color='c')
8 plt.xlabel("p")
9 plt.ylabel("Probability Density")
10 plt.legend()
11 plt.title("Optimized Wavefunction in p-basis")
12 plt.grid(True)
13 plt.show()
```



✓ Answer (end)

✓ d. Coherent States

There is a special wave-function that we will consider later called a coherent state.

The wave-function for coherent states are

$$\Psi(x) = e^{-\frac{1}{2}(x-\alpha\sqrt{2})^2}$$

We will set $\alpha = 0.2$ for this part of the problem.

Using your code, compute the uncertainty ΔX and ΔP and Δ for this (actually will be true for any) coherent state?

Also plot the probability in real and momentum space.

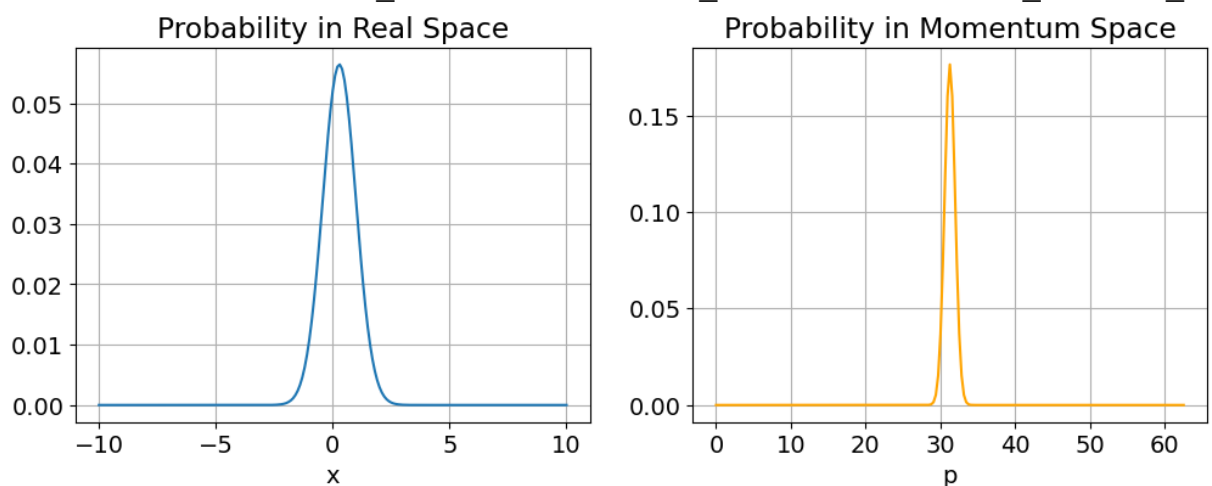
? Answer (start)

```

1 alpha = 0.2
2 psi_coh = np.exp(-0.5 * (xs - alpha * np.sqrt(2))**2)
3 psi_coh = psi_coh / np.linalg.norm(psi_coh)
4
5 dx, dp, dxp = uncertainty(psi_coh, X, P)
6 print(f"Coherent state: delta_X = {np.real(dx):.4f}, delta_P =
7
8 fig, ax = plt.subplots(1, 2, figsize=(12, 4))
9 ax[0].plot(xs, np.abs(psi_coh)**2)
10 ax[0].set_title("Probability in Real Space")
11 ax[0].set_xlabel("x")
12 ax[0].grid(True)
13
14 psi_coh_p = U @ psi_coh
15 ax[1].plot(ps, np.abs(psi_coh_p)**2, color='orange')
16 ax[1].set_title("Probability in Momentum Space")
17 ax[1].set_xlabel("p")
18 ax[1].grid(True)
19 plt.show()

```

Coherent state: delta_X = 0.7071, delta_P = 0.7067, delta_X*delta_P



✓ Answer (end)

✓ e. Not coherent states

So far we've seen examples of states which satiate the uncertainty bound while simultaneously having the same uncertainty in x and p . Verify that this state satiates the Heisenberg uncertainty bound while having different values of ΔX and ΔP

$$\Psi(x) = N \exp[-0.3x^2]$$

where N is the normalization.

? Answer (start)

```
1 psi_nc = np.exp(-0.3 * xs**2)
2 psi_nc = psi_nc / np.linalg.norm(psi_nc)
3
4 dx, dp, dxp = uncertainty(psi_nc, X, P)
5 print(f"Non-coherent state: delta_X = {np.real(dx):.4f}, delta_
```

```
Non-coherent state: delta_X = 0.9129, delta_P = 0.5475, delta_X*delta_P = 0.5
```

✓ Answer (end)

✓ Exercise 2. Harmonic Oscillator in Real Space

The Hamiltonian for the simple harmonic oscillator is

$$H = \frac{\hat{P}^2}{2m} + \frac{1}{2}m\omega^2 \hat{X}^2$$

We know that $\hat{P} = -i\hbar \frac{\partial}{\partial x}$ leaving us with

$$H = -\frac{\hbar^2}{2m} \frac{\partial^2}{\partial x^2} + \frac{1}{2}m\omega^2 x^2$$

Working in units where $\hbar = 1$ and selecting $m = \omega = 1$ we are then left with

$$H = -\frac{1}{2} \frac{\partial^2}{\partial x^2} + \frac{1}{2} x^2$$

✓ a. Hamiltonian and Eigenvalues

Let's start by generating the matrix for this Hamiltonian. We are again going to use $-10 \leq x \leq 10$ via

```
delta_x=0.01
xs=SetupGrid(20,delta_x)
X,P,P2=SetupObservables(xs,delta_x)
```

To generate the Hamiltonian

- The Kinetic piece is as it was in the particle in the box
- The Potential piece is diagonal with $V(x) = 1/2x^2$ down the diagonal.

Plotting the first 10 eigenenergies. You should start trying to make your plots pretty and informative:

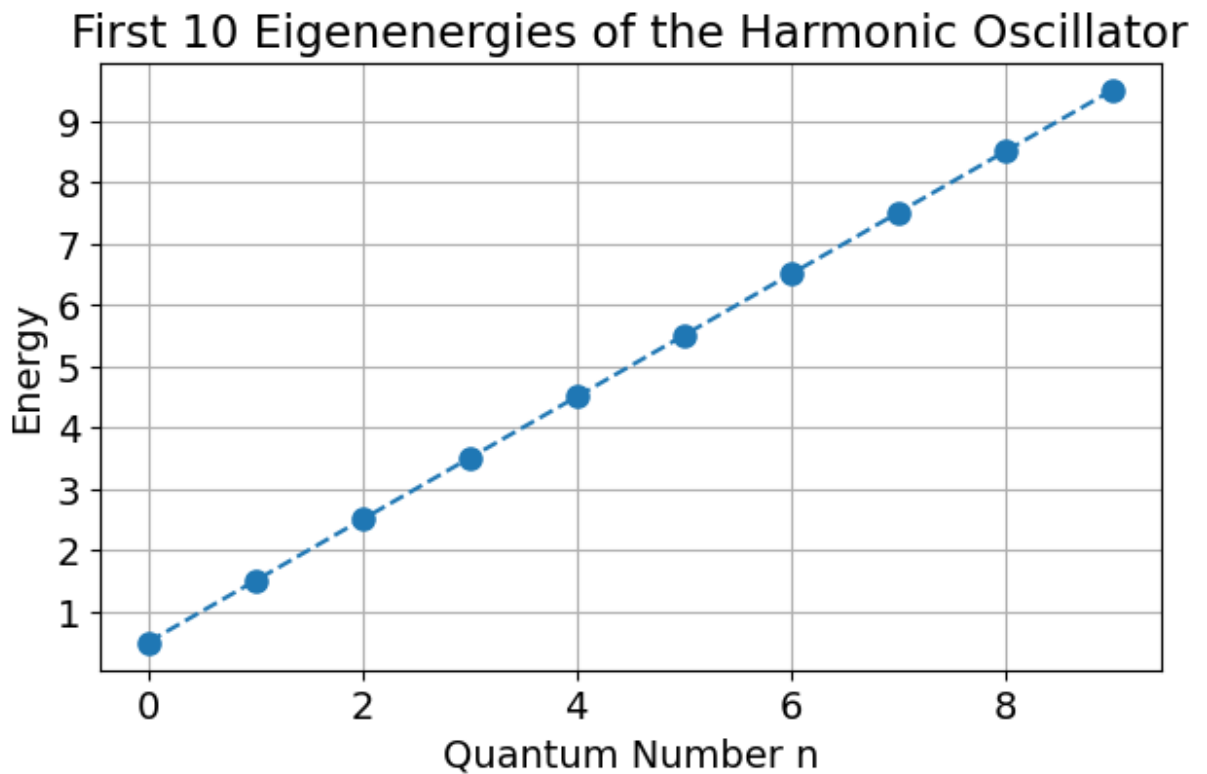
- It's often useful to set up the size of the plot by doing
`fig,ax=plt.subplots(1,1,figsize=(5,4),constrained_layout=True)`
- Include in your plots reasonable markers and sizes
`marker='o',markersize=10,linestyle='--'`
- Turn on a grid (`ax.grid()`)
- Make the grid have a reasonable step
`ax.yaxis.set_major_locator(plt.MultipleLocator(1))`
- Label your axes! (`ax.set_xlabel(...)`)
- Set your limits to something sane (`ax.set_ylim(0,something)`)

What important qualitative feature do you notice about the energy levels?

? Answer (start)

```
1 delta_x = 0.01
2 xs = SetupGrid(20, delta_x)
3 X, P, P2 = SetupObservables(xs, delta_x)
4
5 H = 0.5 * P2 + 0.5 * (X @ X)
6
7 evals, evects = np.linalg.eigh(H)
```

```
1 fig, ax = plt.subplots(1, 1, figsize=(6, 4), constrained_layout
2 ax.plot(range(10), evals[:10], marker='o', markersize=8, linestyle
3 ax.grid(True)
4 ax.yaxis.set_major_locator(plt.MultipleLocator(1))
5 ax.set_xlabel("Quantum Number n")
6 ax.set_ylabel("Energy")
7 ax.set_title("First 10 Eigenenergies of the Harmonic Oscillator
8 plt.show()
9
10 print(" The energy levels are equally spaced.")
```



✓ Answer (end)

✓ b. Eigenstates

Now we want to plot the lowest three eigenstates. You should add labels to your states: in each of your plot commands do

- `ax.plot(x,y,label="Eigenstate ...")` and then include the legend with
- `ax.legend()`

Notice how many nodes (zeros) the n'th eigenstate has.

In class, you learned that the expected solution to the Harmonic Oscillator involves Hermite polynomials. In particular, the n'th eigenstate should be

$$\Psi_n(x) = \frac{1}{\sqrt{2^n n!}} \left(\frac{m\omega}{\pi\hbar} \right)^{1/4} e^{-\frac{m\omega x^2}{2\hbar}} H_n \left(\sqrt{\frac{m\omega}{\hbar}} x \right)$$

where H_n are the Hermite polynomials.

In python to get the Hermite polynomials, use

```
Herm.hermval(xi, herm_coeffs)
```

where $xi = \sqrt{m\omega/\hbar}$ and the `herm_coeffs` need to be a numpy array of size n+1 with a 1 in spot n and 0 otherwise.

- Plot the expected answers on top of your generated answers. You should find that they fall on top of each other.

You may occasionally find that the expected and numerical answer are upside down from each other. This is because the global phase of a wave-function doesn't matter. This means you can always multiply a wave-function by -1 and it's still physically the same. If you find this is happening to you, go ahead and multiply appropriately by -1 to fix it.

- In addition, print out the uncertainty of these wave-functions. Are they anywhere near the Heisenberg limit.

? Answer (start)

```
1 fig, ax = plt.subplots(1, 1, figsize=(12, 8))
2
3 for n in range(3):
4     psi_num = evecs[:, n]
5     # Normalize with respect to dx grid integration
```

```

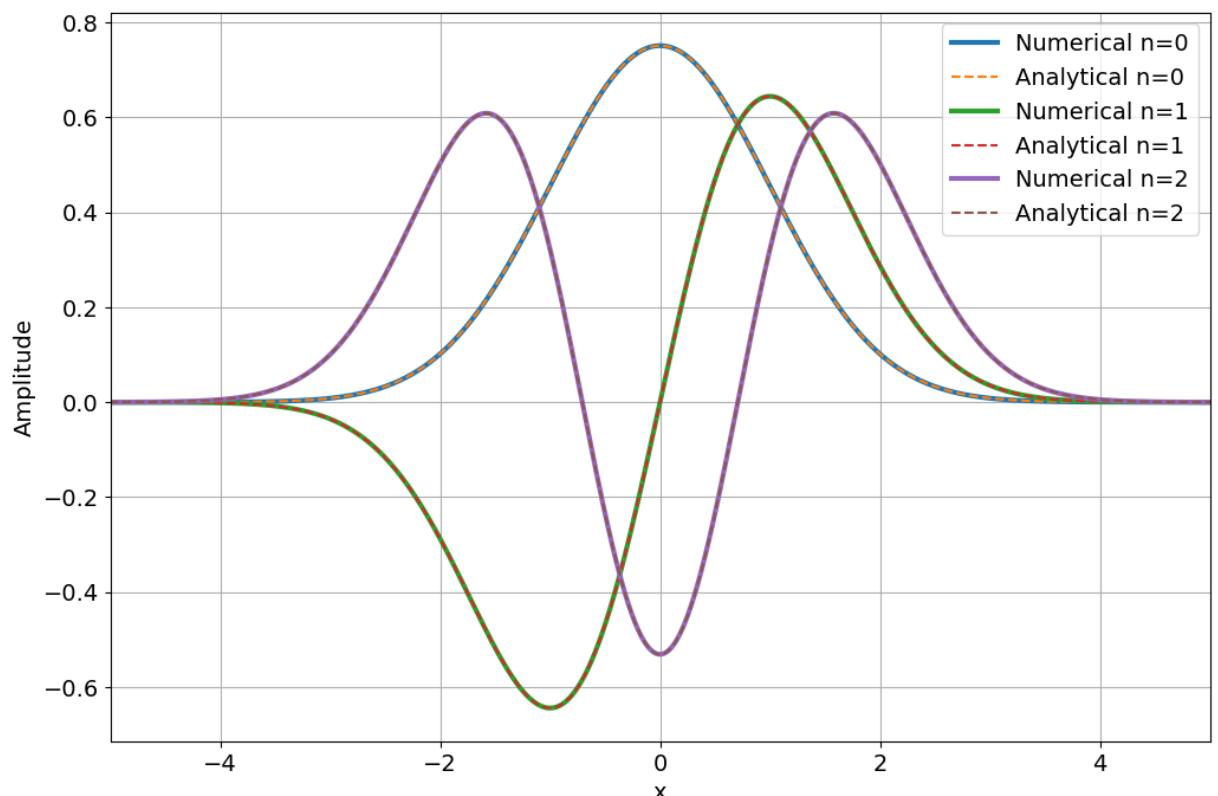
6     psi_num = psi_num / np.sqrt(delta_x) / np.linalg.norm(psi_n
7
8     # Analytical solution using Hermite polynomials
9     herm_coeffs = np.zeros(n + 1)
10    herm_coeffs[n] = 1
11    H_n = Herm.hermval(xs, herm_coeffs)
12    psi_ana = (1.0 / np.sqrt((2**n) * math.factorial(n))) * (1.
13
14    # Global phase correction if upside down
15    if np.sign(psi_num[len(xs)//2 + 5]) != np.sign(psi_ana[len(
16        psi_num = -psi_num
17
18    ax.plot(xs, np.real(psi_num), label=f"Numerical n={n}", lin
19    ax.plot(xs, np.real(psi_ana), '--', label=f"Analytical n={n
20
21    # Calculate uncertainty
22    d_x, d_p, d_xp = uncertainty(evecs[:,n], X, P)
23    print(f"Eigenstate {n}: dx = {np.real(d_x):.4f}, dp = {np.r
24
25    ax.set_xlim(-5, 5)
26    ax.set_xlabel("x")
27    ax.set_ylabel("Amplitude")
28    ax.legend()
29    ax.grid(True)
30    plt.show()

```

Eigenstate 0: dx = 0.7071, dp = 0.7071, dx*dp = 0.5000

Eigenstate 1: dx = 1.2247, dp = 1.2247, dx*dp = 1.5000

Eigenstate 2: dx = 1.5811, dp = 1.5811, dx*dp = 2.5000



✓ Answer (end)

✓ c. Dynamics and Ehrenfest theorem

Like we did in the particle in the box, we are going to do some dynamics.

It's worth encapsulating your time-evolution into two functions (*if you've done the particle-in-a-finite-well assignment you may have already written these functions*)

- `TimeEvolutionOperator(H,delta_t)` should take the Hamiltonian and your time step and return $e^{-iH\delta t}$. It's important to note that for each Hamiltonian, you only need to compute this time-evolution operator once even if you are time-evolving for many steps (or even many different initial wave-functions). This will save significant time compared to computing it each time!
- `TimeEvolution(psi,M,steps)` which takes the initial wave-function `psi`, the time-evolution operator `M` and the number of steps and returns a list of arrays that include the snapshot of the wave-function at every time step.

You will need to give `def update(frame, skip, xs, positions, potential, max_value, energy)` the relevant additional parameters.

`max_value` is the largest probability your wave-function gets (which we've already been calculating for animation) and `energy` is the energy of our initial wave-function, which you can compute (recall the energy doesn't change as a function of time). You also need to send it the position array which you can compute using your `X` from `SetupObservables` (outside the function).

Let's start with the wave-function

- `psi = (np.sqrt(0.1)*v[:,0]+np.sqrt(0.25)*v[:,1]+1.j*np.sqrt(0.65)*v[:,2]).astype(np.complex128)` where `v` are the eigenstates from the SHO.

Perform time-evolution with the initial wave-function with a

- $\delta t = 0.1$ and
- for a total time of $T = 40$.

Store the snapshot of the wave-function at each time and animate it.

? Answer (start)

```

1 def TimeEvolutionOperator(H, delta_t):
2     evals, evecs = np.linalg.eigh(H)
3     exp_D = np.diag(np.exp(-1.j * evals * delta_t))
4     M = evecs @ exp_D @ evecs.conj().T
5     return M
6
7 def TimeEvolution(psi, M, steps):
8     psi_t = [psi]
9     for _ in range(steps):
10         psi_t.append(M @ psi_t[-1])
11     return psi_t

```

```

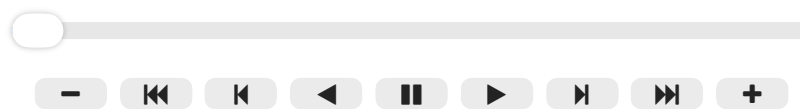
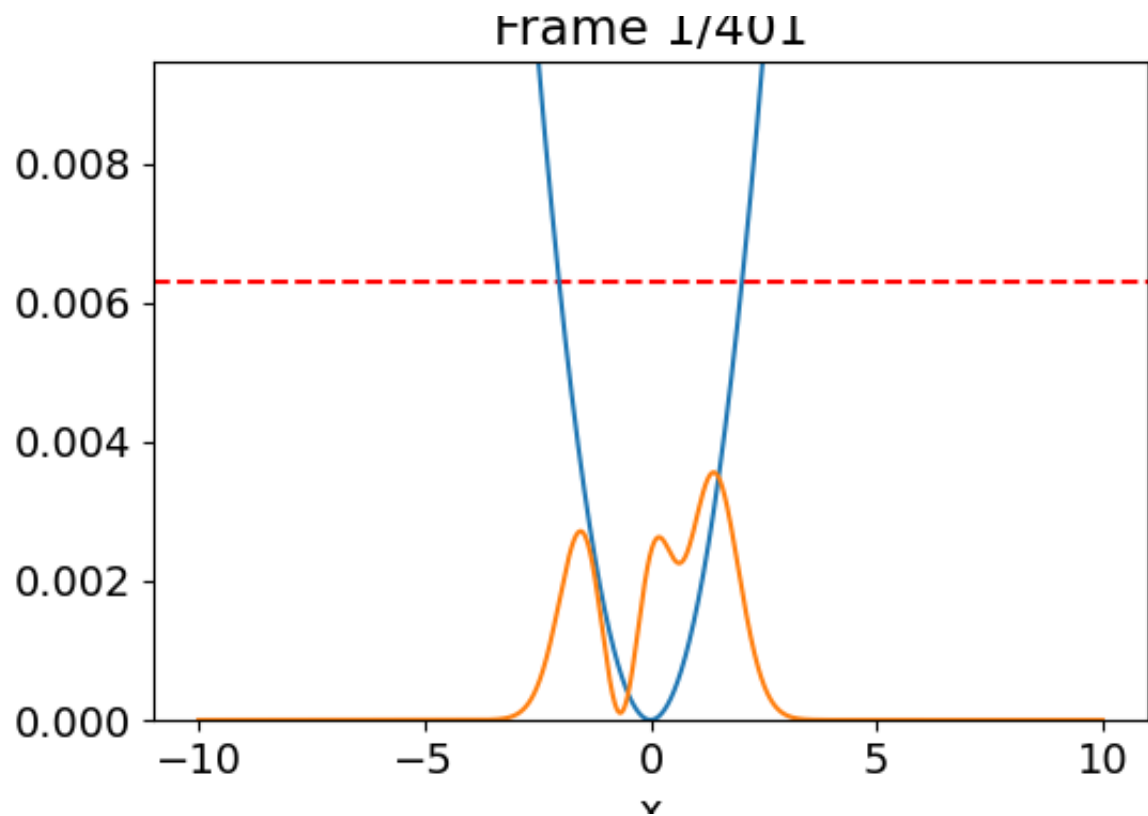
1 psi_init = (np.sqrt(0.1)*evecs[:,0] + np.sqrt(0.25)*evecs[:,1])
2 delta_t = 0.1
3 T = 40
4 steps = int(T / delta_t)
5
6 M = TimeEvolutionOperator(H, delta_t)
7 arrays = TimeEvolution(psi_init, M, steps)
8
9 energy = np.real(psi_init.conj() @ H @ psi_init)
10 max_value = np.max([np.max(np.abs(p)**2) for p in arrays])

```

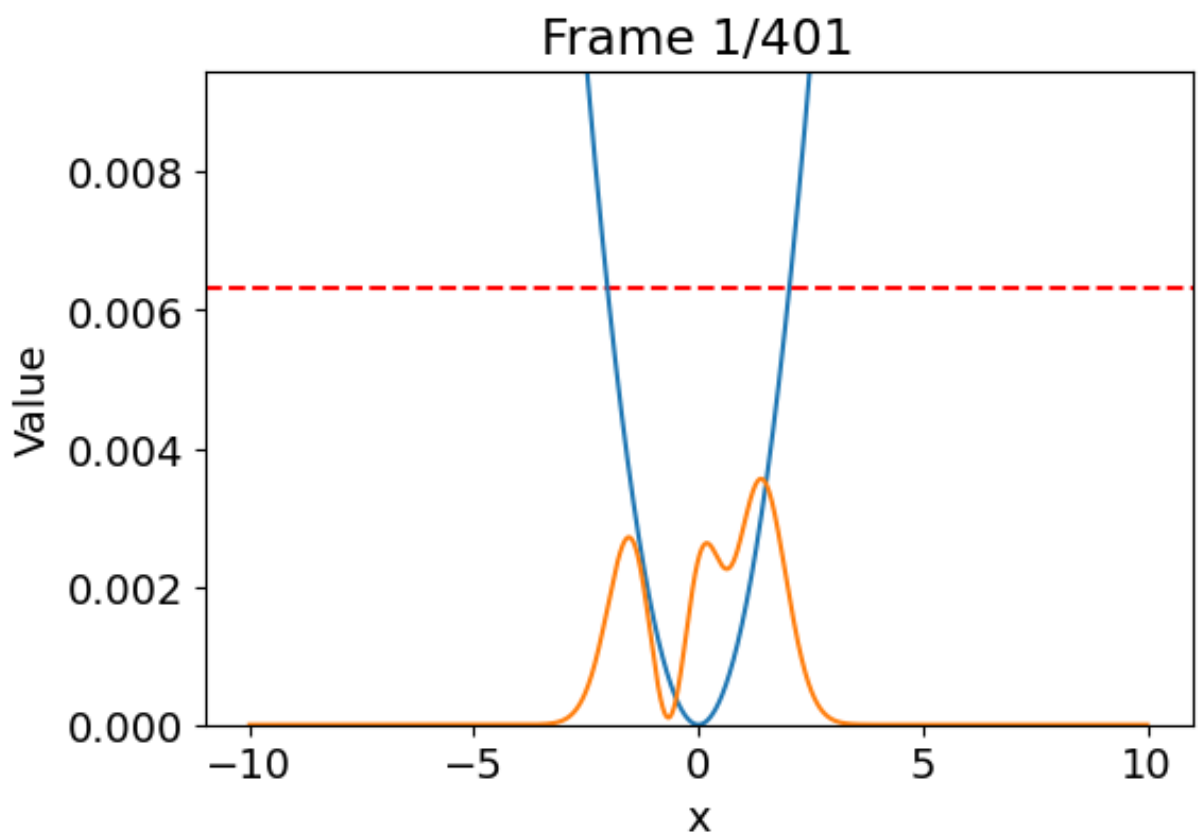
```

1 def update(frame, skip, xs, positions=None, potential=[], max_v
2     plt.cla() # Clear the current plot
3     scale=max_value/energy
4     plt.axhline(energy*scale, color='red', linestyle='--')
5     plt.plot(xs,potential*scale)
6     if type(positions)==np.ndarray:
7         plt.axvline(positions[::skip][frame], color='green', li
8     elif positions!=None:
9         print("ERROR")
10        return
11    plt.plot(xs, np.abs(arrays[::skip][frame])**2) # Plot the
12    plt.ylim(0, 1.5*max_value) # Set the y-axis limit
13    plt.xlabel('x')
14    plt.ylabel('Value')
15    plt.title(f'Frame {skip*frame+1}/{len(arrays)}') # Display
16
17
18 fig = plt.figure(figsize=(6,4))
19 ani = FuncAnimation(fig, update, frames=len(arrays), fargs=(1,
20 HTML(ani.to_jshtml()))
21

```



☐ Once ☒ Loop ☐ Reflect



✓ Answer (end)

As in the particle in the box, takes your list of wave-functions as a function of time and generate from each wave-function a value of $\langle x \rangle(t)$ and $\langle p \rangle(t)$ showing (on the subplots as typical)

On the same plot show

- $\langle x \rangle(t)$
- $\langle p \rangle(t)$
- $\partial \langle x \rangle(t) / \partial t$

What do you notice about the second two plots?

We are also going to add one more subplot this time: `ax=plt.subplots(3,1)`
[1]

For our third subplot what we'd like to plot is

- $\frac{\partial \langle p(t) \rangle}{\partial t}$ generated by finite differences or `np.gradient` using your momentum curve above
- $-\langle V'(x) \rangle(t)$ This is the average of the derivative of the potential over the wave-function. Take the derivative of the potential by hand and then think about what observable you need to get this.

You should find these latter two bullet points are on top of each other.

This (and the previous plots) are a validation of Ehrenfest Theorem which tells us

$$m \frac{d}{dt} \langle x \rangle = \langle p \rangle$$

and

$$\frac{d}{dt} \langle p \rangle = -\langle V'(x) \rangle$$

Finally go ahead and make a phase plot of momentum vs position.

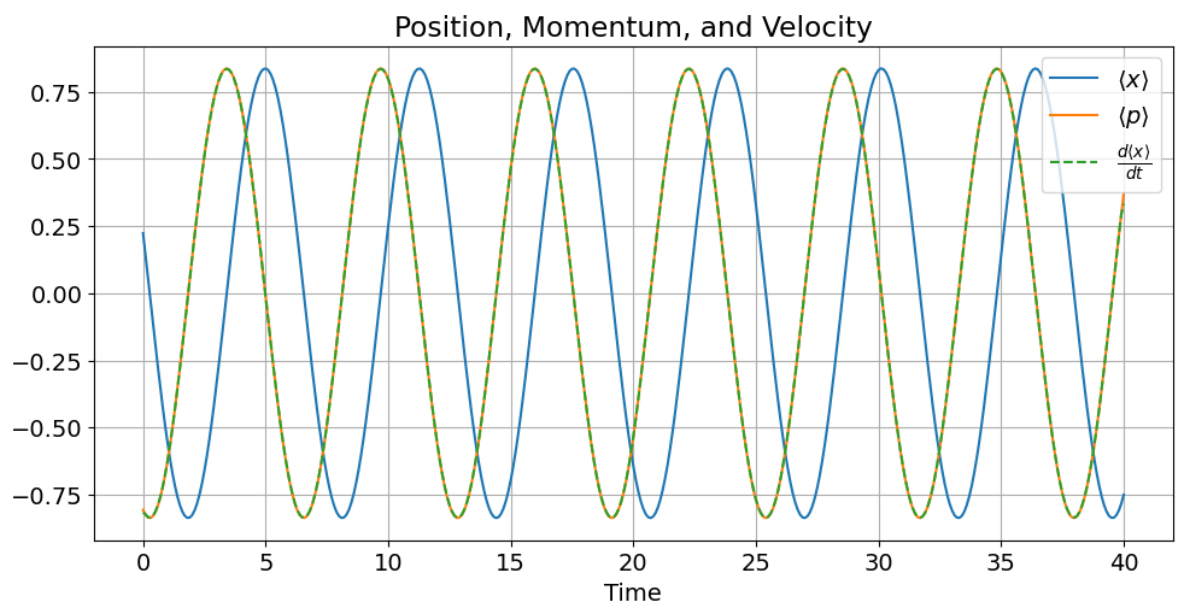
? Answer (start)

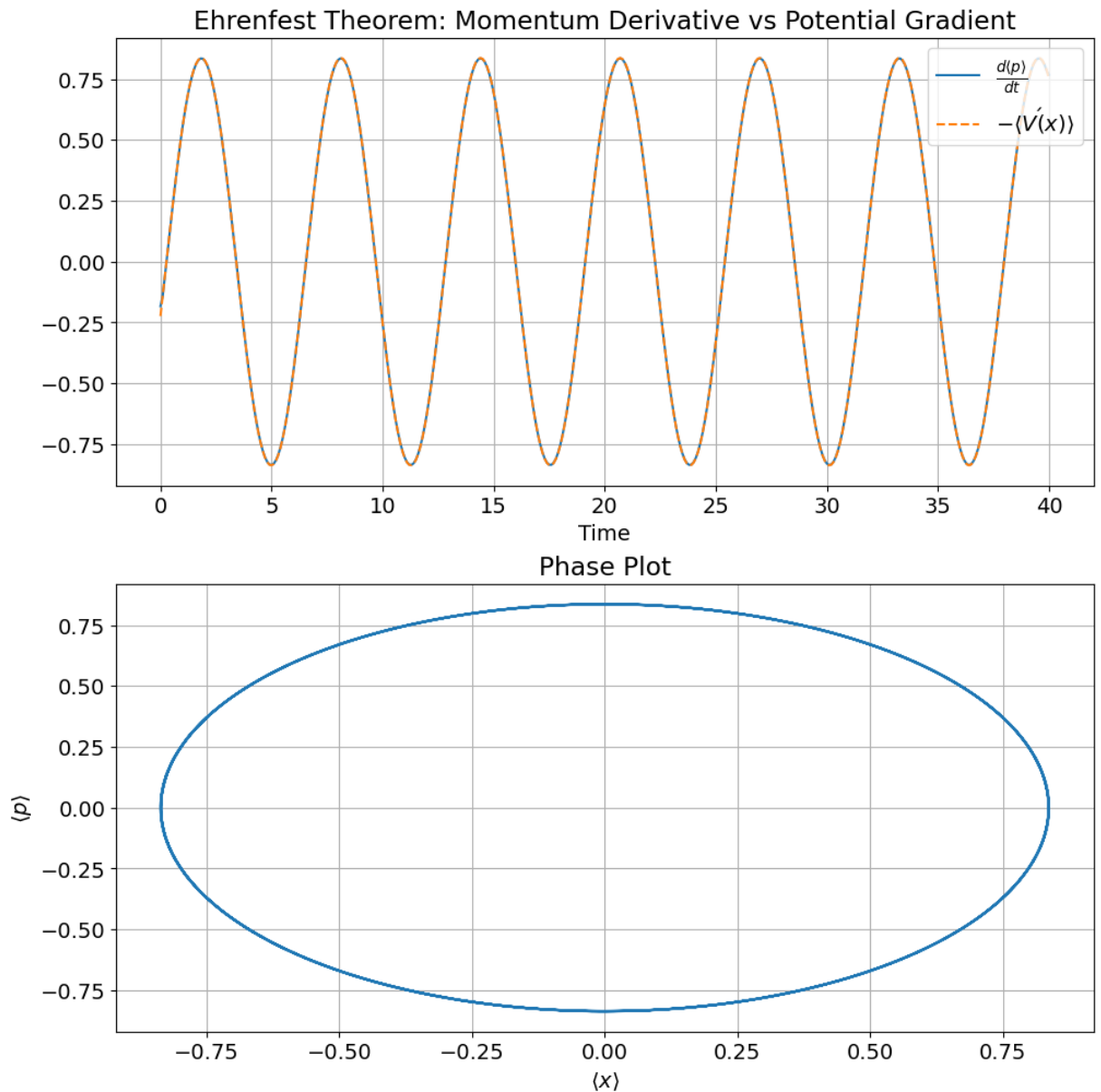
```
1 # Cell 4 for Exercise 2.c - Plotting Ehrenfest theorem
2 t_vals = np.linspace(0, T, steps + 1)
3 x_exp = [np.real(p.conj() @ X @ p) for p in arrays]
4 p_exp = [np.real(p.conj() @ P @ p) for p in arrays]
```

```

5
6 dxdt = np.gradient(x_exp, delta_t)
7 dpdt = np.gradient(p_exp, delta_t)
8
9 #  $V(x) = 0.5 x^2 \rightarrow V'(x) = x \rightarrow -\langle V'(x) \rangle = -\langle x \rangle$ 
10 minus_dV = [-x for x in x_exp]
11
12 fig, ax = plt.subplots(3, 1, figsize=(10, 15), constrained_layout
13
14 ax[0].plot(t_vals, x_exp, label=r'$\langle x \rangle$')
15 ax[0].plot(t_vals, p_exp, label=r'$\langle p \rangle$')
16 ax[0].plot(t_vals, dxdt, '--', label=r'$\frac{d\langle x \rangle}{dt}$')
17 ax[0].legend(loc="upper right")
18 ax[0].set_xlabel("Time")
19 ax[0].set_title("Position, Momentum, and Velocity")
20 ax[0].grid(True)
21
22 ax[1].plot(t_vals, dpdt, label=r'$\frac{d\langle p \rangle}{dt}$')
23 ax[1].plot(t_vals, minus_dV, '--', label=r'$-\langle V'(x) \rangle$')
24 ax[1].legend(loc="upper right")
25 ax[1].set_xlabel("Time")
26 ax[1].set_title("Ehrenfest Theorem: Momentum Derivative vs Poten
27 ax[1].grid(True)
28
29 ax[2].plot(x_exp, p_exp)
30 ax[2].set_xlabel(r'$\langle x \rangle$')
31 ax[2].set_ylabel(r'$\langle p \rangle$')
32 ax[2].set_title("Phase Plot")
33 ax[2].grid(True)
34
35 plt.show()
36
37 print("Notice that  $d\langle x \rangle/dt$  and  $\langle p \rangle$  lie almost exactly on top of

```





Notice that $d\langle x \rangle/dt$ and $\langle p \rangle$ lie almost exactly on top of each other.

✓ Answer (end)

✓ d. Coherent States

We now wish to run time dynamics on a coherent state. Recall the coherent state (as above) is

$$\Psi(x) = \frac{1}{N} e^{-\frac{1}{2}(x-\alpha\sqrt{2})^2}$$

where N normalizes the wave-function. We will use $\alpha = 3$

Do time-dynamics on this system generating both the animation as well as the other plots above.

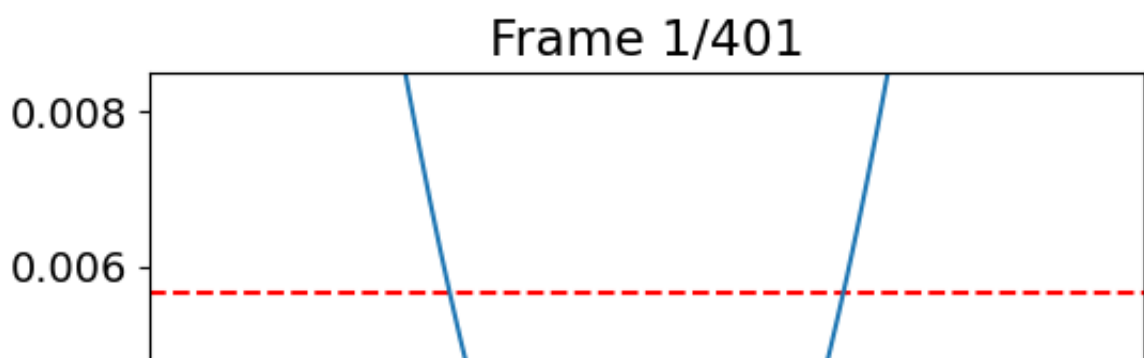
Add one more line to your plots: the position and velocity of a classical oscillator in the same Harmonic potential starting at $x = 3\sqrt{2}$

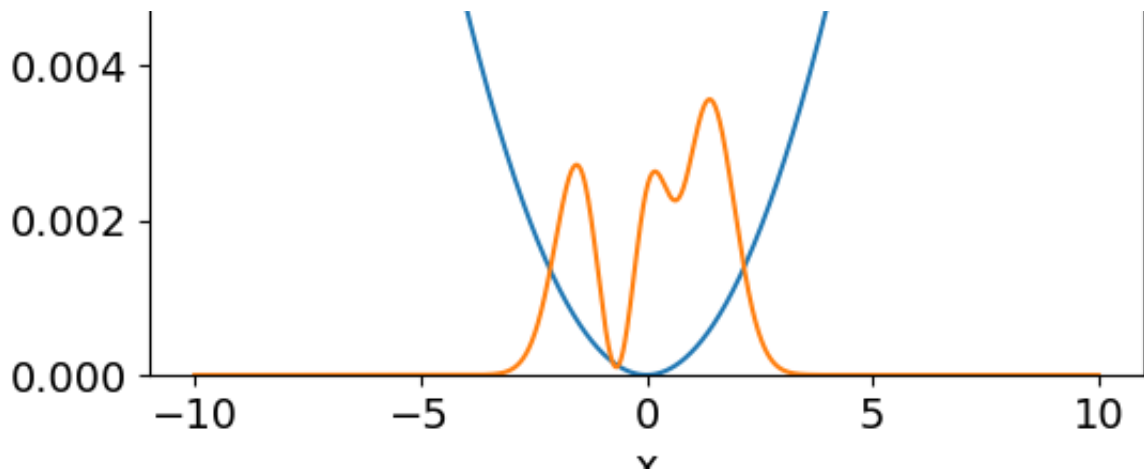
What do you notice that's interesting about the time-evolution of the coherent states.

Does something not quite look right in your animation?

? Answer (start)

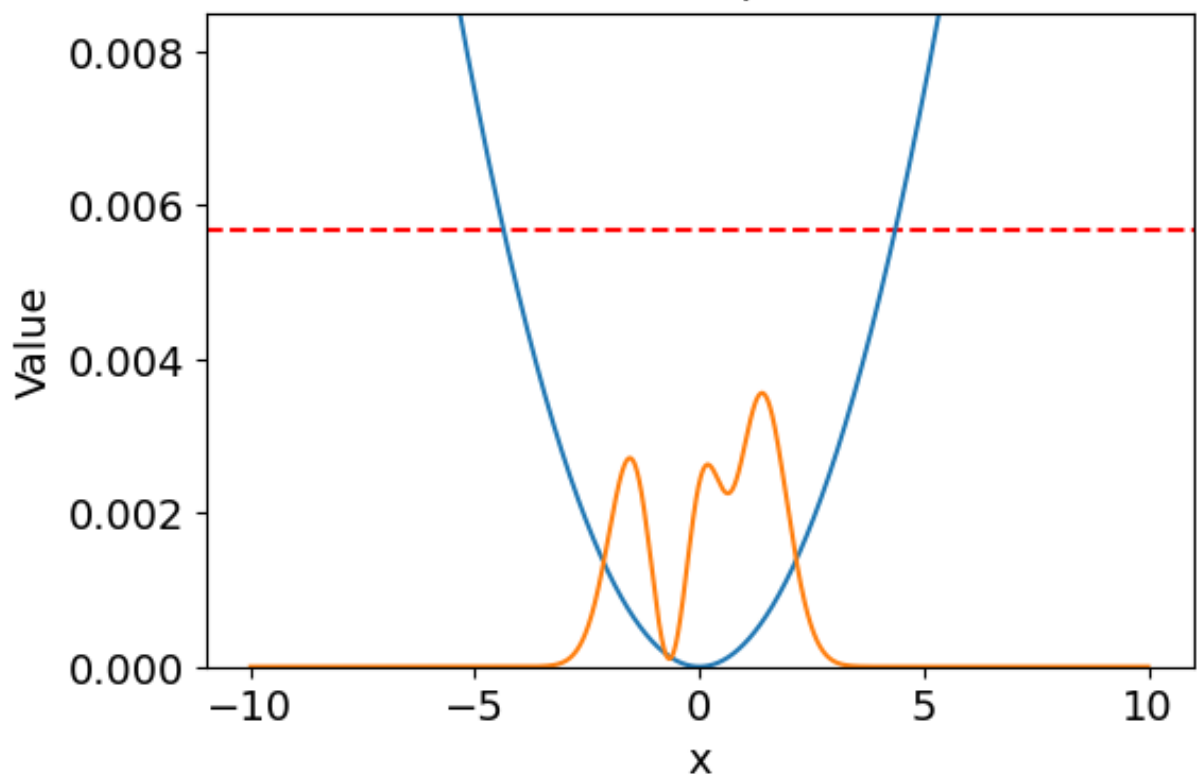
```
1 alpha_c = 3
2 psi_coh2 = np.exp(-0.5 * (xs - alpha_c * np.sqrt(2))**2).astype
3 psi_coh2 = psi_coh2 / np.linalg.norm(psi_coh2)
4
5 arrays_coh = TimeEvolution(psi_coh2, M, steps)
6 energy_coh = np.real(psi_coh2.conj() @ H @ psi_coh2)
7 max_value_coh = np.max([np.max(np.abs(p)**2) for p in arrays_coh])
8
9 fig = plt.figure(figsize=(6,4))
10 ani2 = FuncAnimation(fig, update, frames=len(arrays_coh), fargs=
11 HTML(ani2.to_jshtml()))
12
```





☐ Once ☒ Loop ☐ Reflect

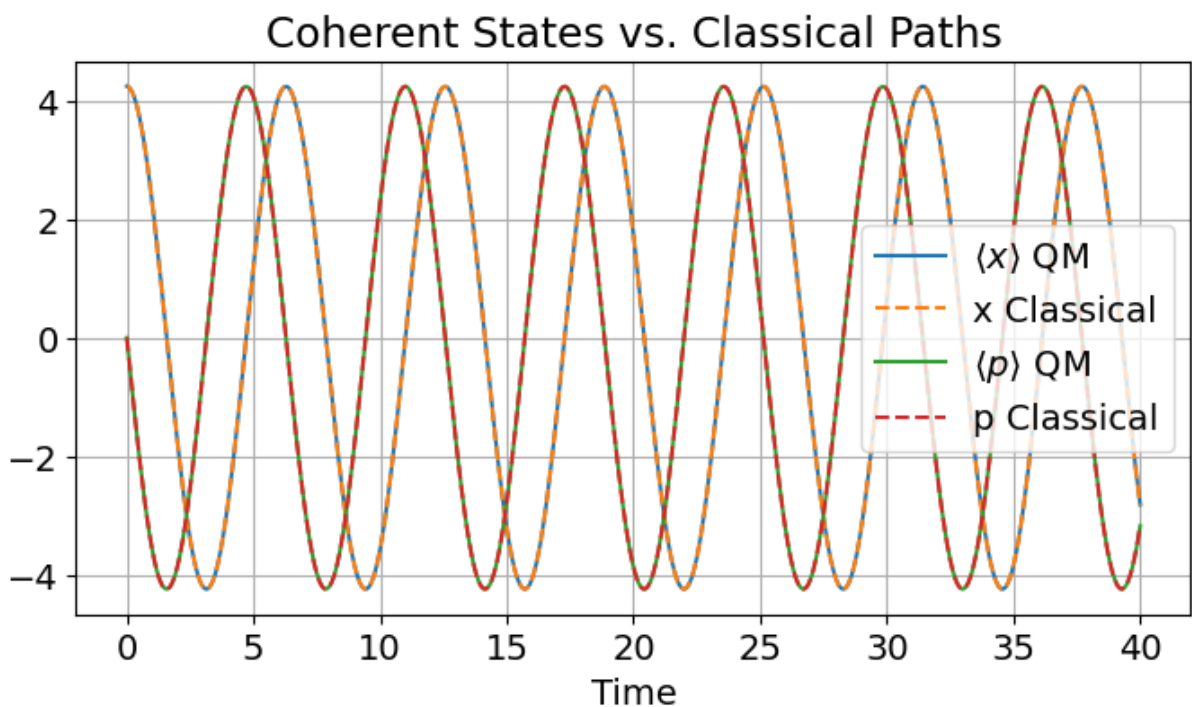
Frame 1/401



```

1 # Cell 2 for Exercise 2.d – Comparison plot
2 x_exp_coh = [np.real(p.conj() @ X @ p) for p in arrays_coh]
3 p_exp_coh = [np.real(p.conj() @ P @ p) for p in arrays_coh]
4
5 # Classical oscillator:  $x(t) = A \cos(\omega t + \phi)$ 
6 x_class = 3 * np.sqrt(2) * np.cos(t_vals)
7 v_class = -3 * np.sqrt(2) * np.sin(t_vals)
8
9 fig, ax = plt.subplots(1, 1, figsize=(8, 4))
10 ax.plot(t_vals, x_exp_coh, label=r'$\langle x \rangle$ QM')
11 ax.plot(t_vals, x_class, '--', label='x Classical')
12 ax.plot(t_vals, p_exp_coh, label=r'$\langle p \rangle$ QM')
13 ax.plot(t_vals, v_class, '--', label='p Classical')
14 ax.legend()
15 ax.set_xlabel("Time")
16 ax.grid(True)
17 plt.title("Coherent States vs. Classical Paths")
18 plt.show()
19
20 print("The quantum coherent state exact replicates the classica

```



The quantum coherent state exact replicates the classical trajectory

The quantum coherent state exact replicates the classical trajectory and maintains its shape over time.

✓ Answer (end)

✓ Exercise 3. Raising and Lowering Operators

✓ a. Raising and Lowering Operators

With the Harmonic oscillator, we've learned that it is useful to look at raising and lowering operators.

$$a = \sqrt{\frac{m\omega}{2\hbar}} \left(\hat{X} + \frac{i}{m\omega} \hat{P} \right)$$

$$a^\dagger = \sqrt{\frac{m\omega}{2\hbar}} \left(\hat{X} - \frac{i}{m\omega} \hat{P} \right)$$

Build both $a^\dagger$ and a in the real space basis.

- Verify that the operators are complex conjugates of each other (by daggering a and subtracting $a^\dagger$ and making sure they are zero - i.e. check that the maximum of the absolute value of the difference is zero)
- Look at the matshow of the upper left of the matrix `plt.matshow(a[0:10,0:10])`. Can you understand why this is what this looks like.

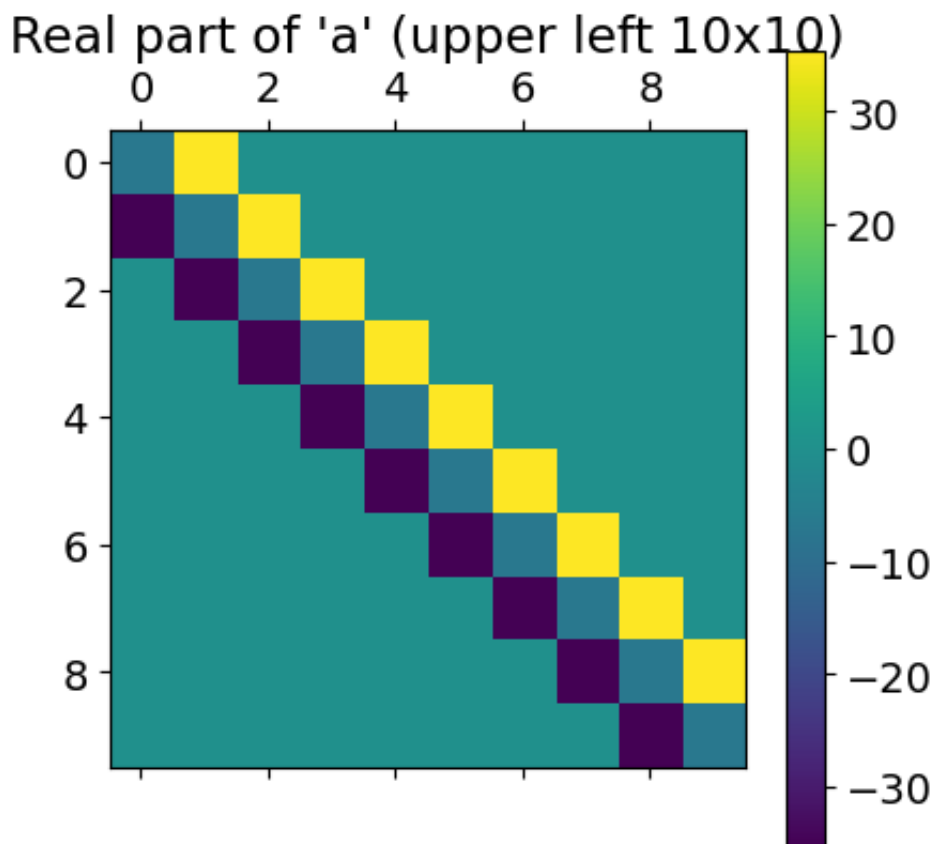
? Answer (start)

```

1 a = (X + 1.j * P) / np.sqrt(2)
2 a_dag = (X - 1.j * P) / np.sqrt(2)
3
4 diff = np.max(np.abs(a.conj().T - a_dag))
5 print(f"Maximum difference between a^dagger and (a)^dagger: {diff}")
6
7 plt.matshow(np.real(a[0:10, 0:10]))
8 plt.title("Real part of 'a' (upper left 10x10)")
9 plt.colorbar()
10 plt.show()
11 print("It looks like this because X is purely diagonal and P is

```

Maximum difference between a^dagger and (a)^dagger: 0.0



It looks like this because X is purely diagonal and P is purely off-

✓ Answer (end)

✓ b. Testing the raising of the raising operator

We expect that the raising operator has the property that when it acts on an eigenstate $|n - 1\rangle$ it should produce the eigenstate $\sqrt{n}$ times eigenstate $|n\rangle$. Let's check that by plotting. Plot $\sqrt{4}|\Psi_4\rangle$ and $a^\dagger|\Psi_3\rangle$ and verify that they give the same result. Again be cognizant that you might need to flip one of the states.

Similarly let's check that a lowers a state. Verify that eigenstate 4 is the same as eigenstate 5 lowered with a coefficient of $\sqrt{5}$.

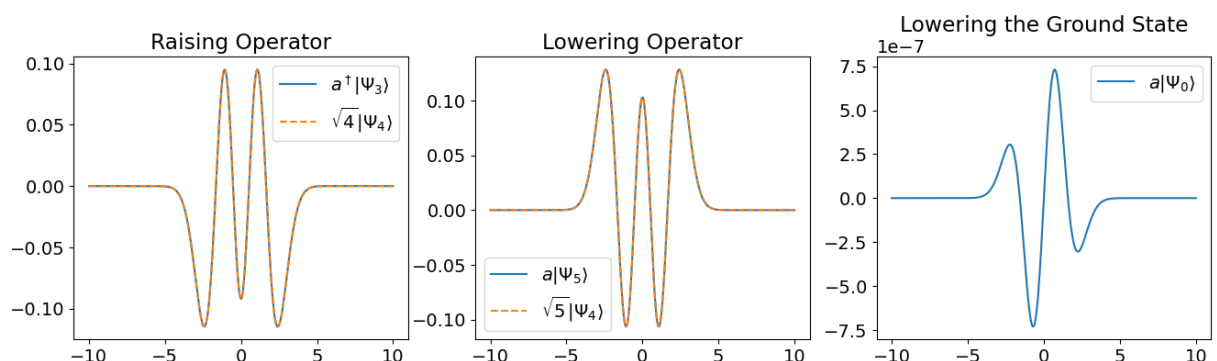
Also verify that the lowering operator destroys the ground state - i.e. check that $a^\dagger\Psi_0$ is actually zero. (Remember to check the scale of your graph as it might have some features but essentially be zero)

? Answer (start)

```

1 psi_3 = evecs[:, 3]
2 psi_4 = evecs[:, 4]
3 psi_5 = evecs[:, 5]
4
5 raised_3 = a_dag @ psi_3
6 expected_4 = np.sqrt(4) * psi_4
7
8 if np.sign(np.real(raised_3[len(xs)//2+5])) != np.sign(np.real(
9     expected_4 = -expected_4
10
11 fig, ax = plt.subplots(1, 3, figsize=(16, 4))
12 ax[0].plot(xs, np.real(raised_3), label=r'$a^\dagger |\Psi_3\rangle$')
13 ax[0].plot(xs, np.real(expected_4), '--', label=r'$\sqrt{4} |\Psi_4\rangle$')
14 ax[0].legend()
15 ax[0].set_title("Raising Operator")
16
17 lowered_5 = a @ psi_5
18 expected_4_low = np.sqrt(5) * psi_4
19
20 if np.sign(np.real(lowered_5[len(xs)//2+5])) != np.sign(np.real(
21     expected_4_low = -expected_4_low
22
23 ax[1].plot(xs, np.real(lowered_5), label=r'$a |\Psi_5\rangle$')
24 ax[1].plot(xs, np.real(expected_4_low), '--', label=r'$\sqrt{5} |\Psi_4\rangle$')
25 ax[1].legend()
26 ax[1].set_title("Lowering Operator")
27
28 lowered_0 = a @ evecs[:, 0]
29 ax[2].plot(xs, np.real(lowered_0), label=r'$a |\Psi_0\rangle$')
30 ax[2].legend()
31 ax[2].set_title("Lowering the Ground State")
32 plt.show()

```



Note that scale of the graph on the right is $1e-7$ which means wavefunction is essentially zero as expected. Lowering ground state yields zero.

✓ Answer (end)

✓ c. Verifying the norms and number operators

Now we would like to further verify that the raising and lowering operators always act with the additional $\sqrt{N}$ term. Since our eigenstates start normalized, if we compute the norm (`np.linalg.norm`) of the eigenstates after they've been hit by the raising and lowering operators you should find that their norm is $\sqrt{N}$. Verify this by

- plotting the norm of the eigenstates (separately for the raising and lower operator) hitting the first ten eigenstates
- plotting $\sqrt{N}$

and checking that they are the same

We can also check that $a^\dagger a$ is a number operator. Plot again for the first ten eigenstates, the expectation value of $a^\dagger a$ in those eigenstates.

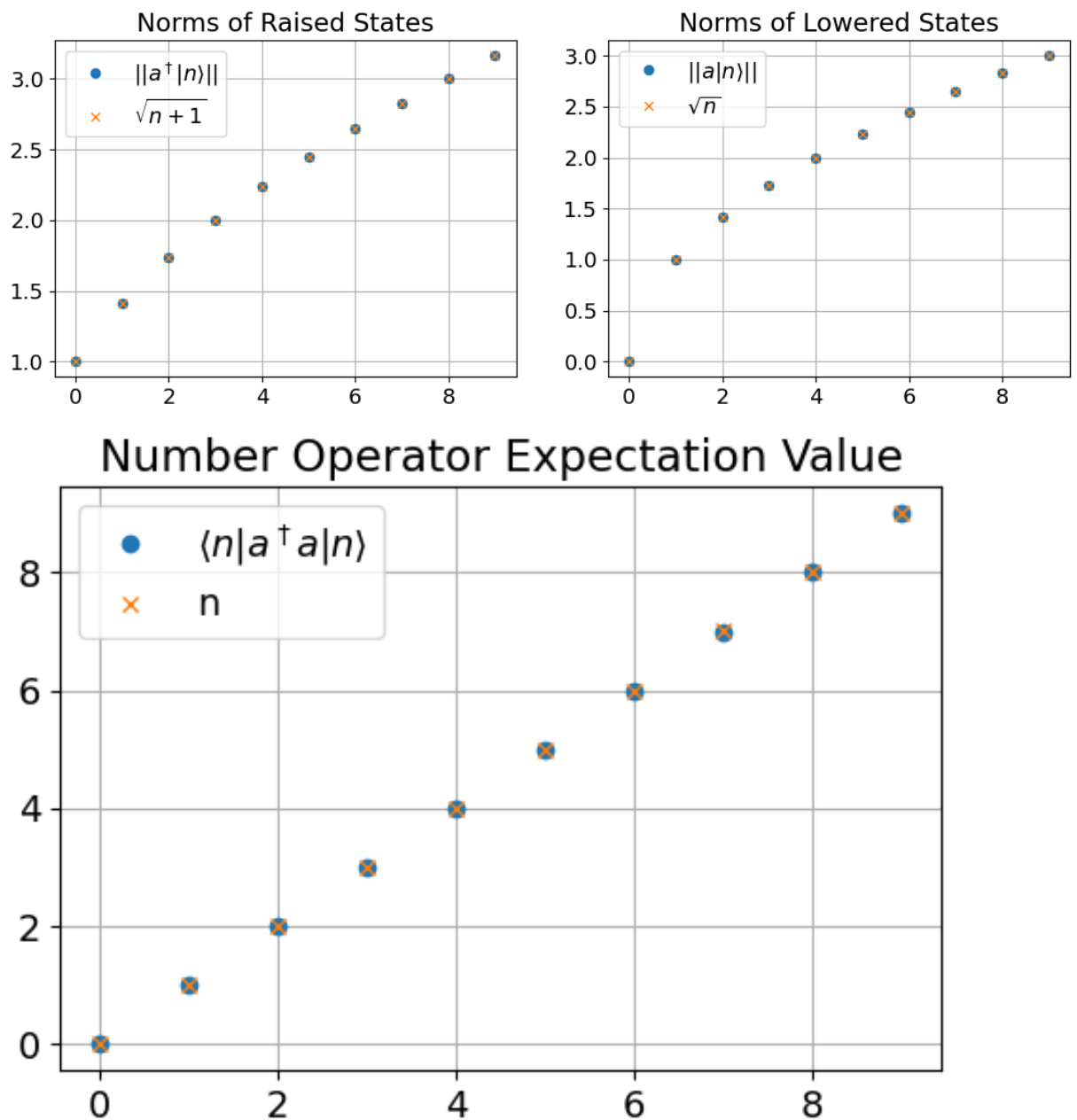
? Answer (start)

```
1 norms_raised = [np.linalg.norm(a_dag @ evects[:, n]) for n in range(10)]
2 norms_lowered = [np.linalg.norm(a @ evects[:, n]) for n in range(10)]
3 sqrt_n_plus_1 = [np.sqrt(n + 1) for n in range(10)]
4 sqrt_n = [np.sqrt(n) for n in range(10)]
5
6 fig, ax = plt.subplots(1, 2, figsize=(12, 4))
7 ax[0].plot(range(10), norms_raised, 'o', label=r'$|| a^\dagger$')
8 ax[0].plot(range(10), sqrt_n_plus_1, 'x', label=r'$\sqrt{n+1}$')
9 ax[0].legend()
10 ax[0].grid(True)
11 ax[0].set_title("Norms of Raised States")
12
13 ax[1].plot(range(10), norms_lowered, 'o', label=r'$|| a$')
14 ax[1].plot(range(10), sqrt_n, 'x', label=r'$\sqrt{n}$')
15 ax[1].legend()
16 ax[1].grid(True)
17 ax[1].set_title("Norms of Lowered States")
```

```

18 plt.show()
19
20 num_op = a_dag @ a
21 exp_num = [np.real(evecs[:, n].conj() @ num_op @ evecs[:, n]) f
22
23 plt.figure(figsize=(6, 4))
24 plt.plot(range(10), exp_num, 'o', label=r'$\langle n | a^\dagger a | n \rangle$')
25 plt.plot(range(10), range(10), 'x', label='n')
26 plt.legend()
27 plt.grid(True)
28 plt.title("Number Operator Expectation Value")
29 plt.show()

```



✓ Answer (end)

✓ d. Write it in the basis of number operators

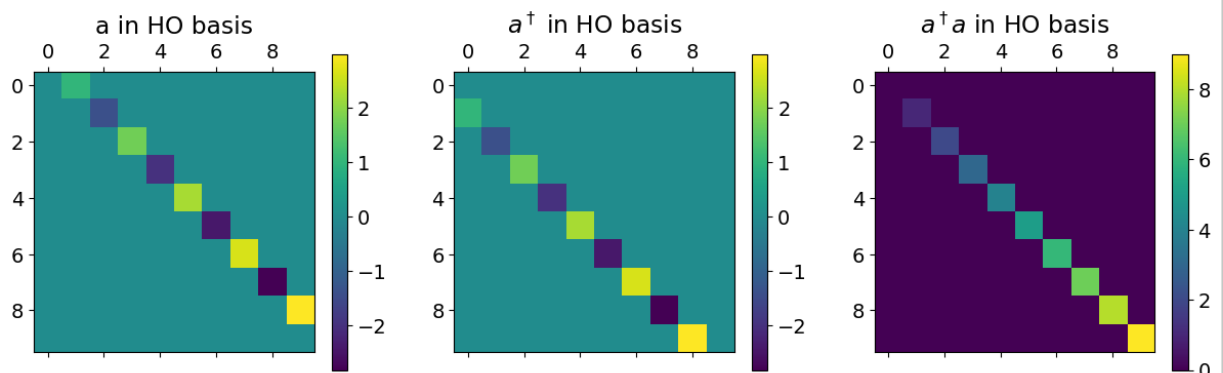
Now we would like to go ahead and write $a^\dagger$ and a in the Harmonic Oscillator basis. In this basis, what we expect to see is that both the raising and lowering operators are just above and below the diagonal respectively. To rotate an operator from the position basis into the Harmonic oscillator basis, you want to do $v.T.conjugate() @ 0 @ v$ for an operator 0 where v is the eigenstates of the simple harmonic oscillator. Go ahead and rotate a , $a^\dagger$ and $a^\dagger a$ into the Harmonic oscillator basis and look at the top 10×10 chunk. Does it make sense? What are the values of the diagonal of $a^\dagger a$

? Answer (start)

```

1 v = evects
2 a_H0 = v.conj().T @ a @ v
3 a_dag_H0 = v.conj().T @ a_dag @ v
4 num_H0 = v.conj().T @ num_op @ v
5
6 fig, ax = plt.subplots(1, 3, figsize=(15, 4))
7
8 cax0 = ax[0].matshow(np.real(a_H0[:10, :10]))
9 ax[0].set_title("a in H0 basis")
10 fig.colorbar(cax0, ax=ax[0])
11
12 cax1 = ax[1].matshow(np.real(a_dag_H0[:10, :10]))
13 ax[1].set_title(r"$a^\dagger$ in H0 basis")
14 fig.colorbar(cax1, ax=ax[1])
15
16 cax2 = ax[2].matshow(np.real(num_H0[:10, :10]))
17 ax[2].set_title(r"$a^\dagger a$ in H0 basis")
18 fig.colorbar(cax2, ax=ax[2])
19
20 plt.show()

```



The diagonal of $a^\dagger a$ maps exactly to 0, 1, 2, ..., 9. It makes total sense because these constitute the exact eigenvalues of the underlying number operator.

✓ Answer (end)

✓ e. Warts

So far everything has worked out well when we discretized things. In this section, we just mention a couple warts that you need to look out for.

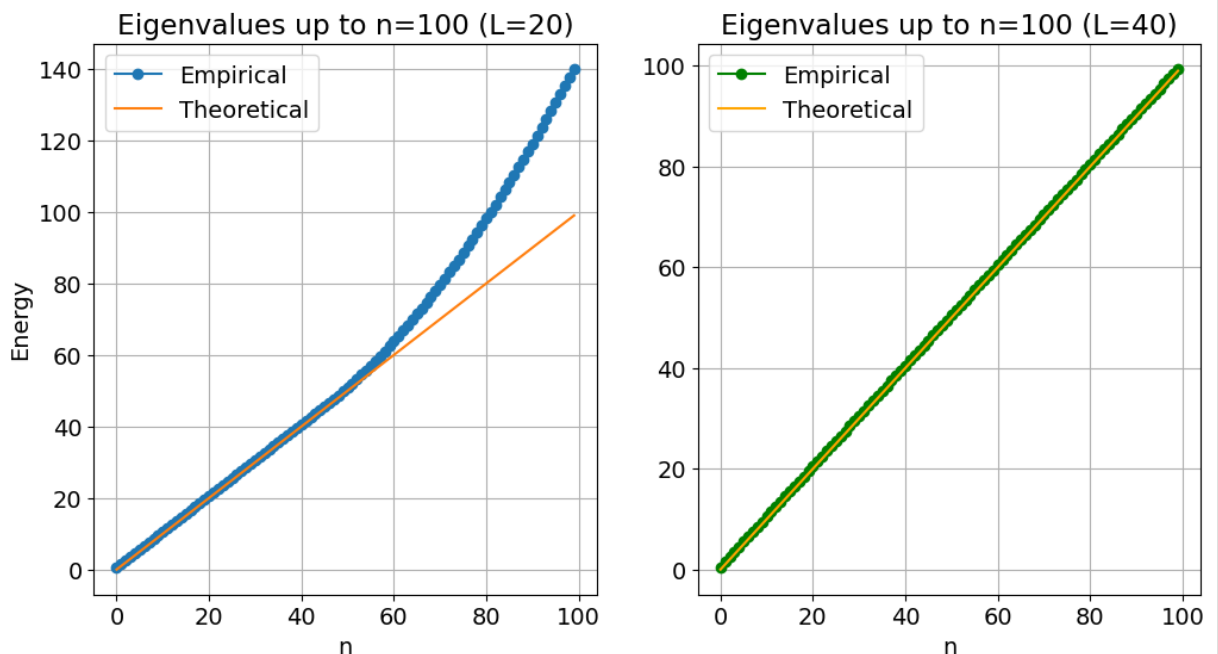
- The simplest thing is just that the eigenstates and eigenvalues start diverging from the true answer due to the finite L and discretization. This is very standard but you can see this here by plotting the eigenvalues out to $n = 100$. What do you notice? To fix this, you can go ahead and increase $L = 40$ and see that the eigenstates out to $n = 100$ look fine.

? Answer (start)

```

1 # Cell 1 for Exercise 3.e - Changing the grid size L
2 fig, ax = plt.subplots(1, 2, figsize=(12, 6))
3 ax[0].plot(range(100), evals[:100], 'o-' )
4 ax[0].plot(range(100), range(100), '-')
5 ax[0].set_title("Eigenvalues up to n=100 (L=20)")
6 ax[0].set_xlabel("n")
7 ax[0].set_ylabel("Energy")
8 ax[0].legend(["Empirical", "Theoretical"])
9 ax[0].grid(True)
10
11 xs_40 = SetupGrid(40, delta_x)
12 X_40, P_40, P2_40 = SetupObservables(xs_40, delta_x)
13 H_40 = 0.5 * P2_40 + 0.5 * (X_40 @ X_40)
14 evals_40 = np.linalg.eigvalsh(H_40)
15
16 ax[1].plot(range(100), evals_40[:100], 'o-', color='green')
17 ax[1].plot(range(100), range(100), '-', color='orange')
18 ax[1].set_title("Eigenvalues up to n=100 (L=40)")
19 ax[1].set_xlabel("n")
20 ax[1].legend(["Empirical", "Theoretical"])
21 ax[1].grid(True)
22 plt.show()
23
24

```



With $L=20$, the eigenvalues diverge from the linear trend for higher quantum numbers because the physical dimensions of the wider wavefunctions spread out beyond the boundary of the simulated grid.

✓ Answer (end)

We learned in quantum mechanics that x and p obey canonical commutation relations - i.e. $[X, P] = i\hbar$ (which means explicitly it's the identity matrix times $i\hbar$).

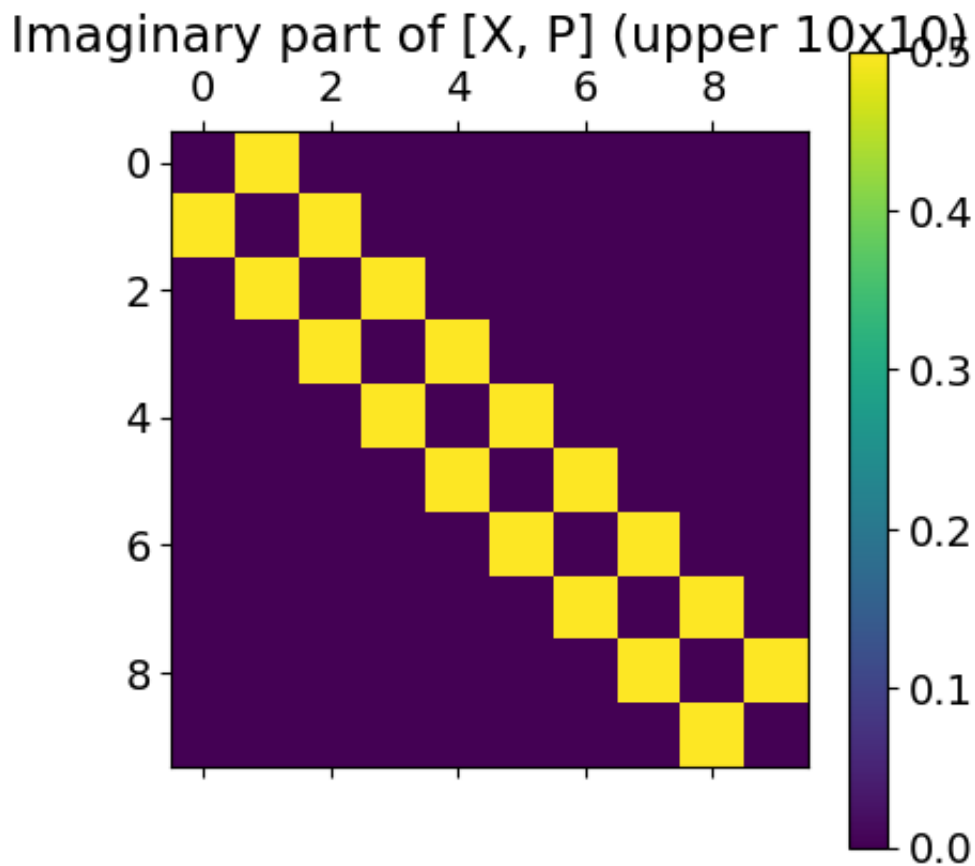
Let's test this. Take the X and P operators and compute their commutator. Do a matshow of the imaginary part of the top 10×10 piece of the matrix.

? Answer (start)

```

1 # Cell 2 for Exercise 3.e - Commutators and Fermion Doubling
2 comm = X @ P - P @ X
3 plt.matshow(np.imag(comm[:10, :10]))
4 plt.title("Imaginary part of [X, P] (upper 10x10)")
5 plt.colorbar()
6 plt.show()
7

```



✓ Answer (end)

You may notice that you get something that is not diagonal at all. Instead there is $i\hbar$ right off the diagonal. What's going on!?

Remember that the rows and columns are labelled by x . This means that what we expected was

$$\langle x | [\hat{X}, \hat{P}] | x \rangle = i\hbar$$

But what we got was the piece that was just off the diagonal having $i\hbar$ on it - i.e. you found that

$$\langle x | [\hat{X}, \hat{P}] | x + \Delta x \rangle = \langle x + \Delta x | [\hat{X}, \hat{P}] | x \rangle = i\hbar$$

This is very close especially since you could take Δx to be very very tiny. So we almost got the right thing.

What went wrong?

When you actually have finite matrices the trace of a commutator always has to be zero:

$$\text{Tr}(XP - PX) = \text{Tr}(XP) - \text{Tr}(PX) = \text{Tr}(XP) - \text{Tr}(XP) = 0$$

where the matrices in the trace got flipped because the trace is invariant with respect to cycling matrices. So it's just impossible to have $i\hbar$ down the diagonal. That said, given the closeness it doesn't cause us much trouble.

- Finally, we have learned that

$$H = \frac{1}{2} + \hbar\omega a a^\dagger$$

is the Harmonic Oscillator Hamiltonian.

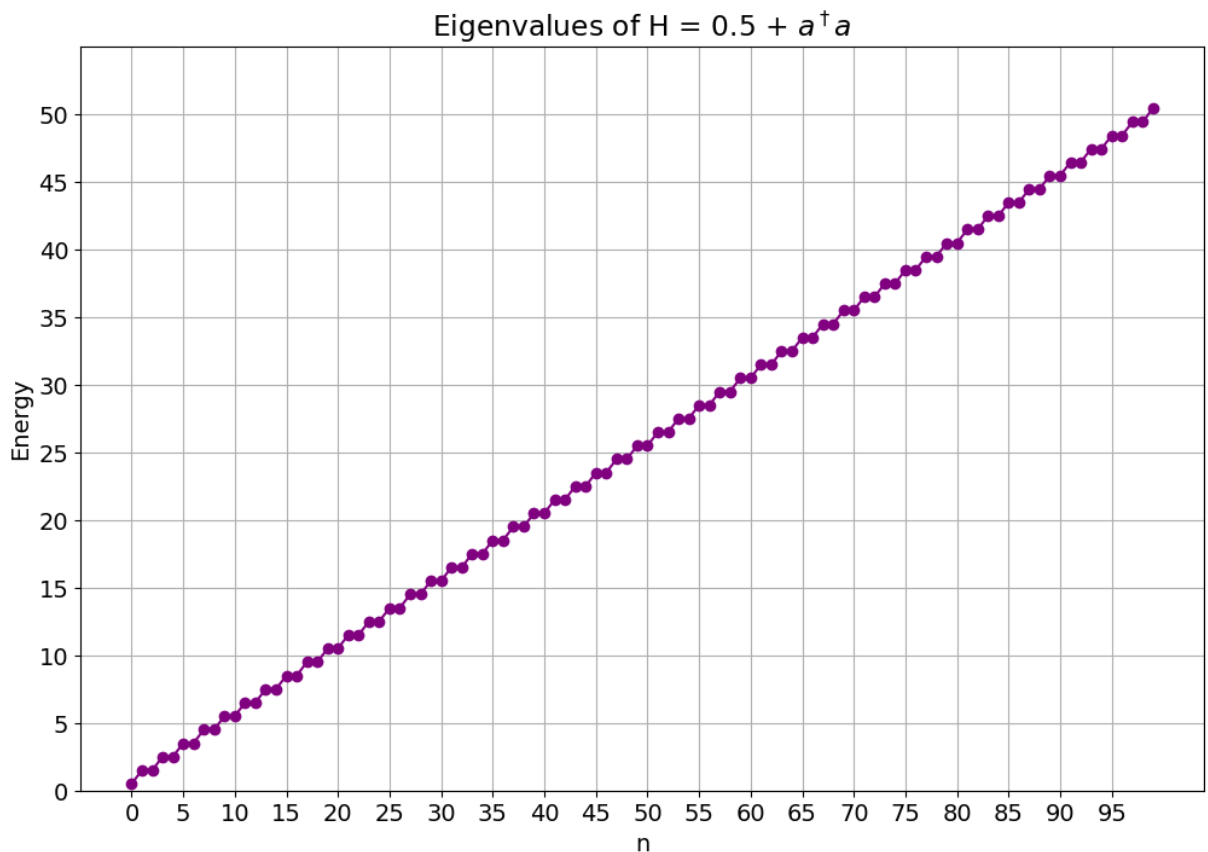
Go ahead and diagonalize this in the x -basis and plot the first 100 eigenvalues.

You'll notice a set of eigenvalues that you are used to (which can correspond to the eigenvectors you are used to) and an erroneous set of eigenvalues. This is essentially coming from the fact that $P@P$ and P^2 are different matrices. You might think that this is some technical issue but it's actually deep and fundamental and causes all sorts of trouble in lattice gauge theory. It goes by the name Fermion doubling and you can see a description of the problem (and the piece that's very relevant here) if you look at this [Wikipedia article](#) and look at the section on derivative discretization.

? Answer (start)

```
1 H_alt = 0.5 * np.eye(len(xs)) + a_dag @ a
2 evals_alt = np.linalg.eigvalsh(H_alt)
```

```
1 plt.figure(figsize=(12, 8))
2 plt.plot(range(100), evals_alt[:100], 'o-', color='purple')
3 plt.title(r"Eigenvalues of H = 0.5 + $a^\dagger a$")
4 plt.ylim([0, 55])
5 plt.xticks(range(0,100,5))
6 plt.yticks(range(0,55,5))
7 plt.xlabel("n")
8 plt.ylabel("Energy")
9 plt.grid(True)
10 plt.show()
```



✓ Answer (end)

Acknowledgements:

- Bryan Clark (original)

© Copyright 2025

Double-click (or enter) to edit